

פרק 5

פרק 5.1

בפרק הקודם (Chapter 3) למדת את הבסיס של ספריית  Datasets, והיו 3 שלבים עיקריים ל-fine-tuning:

1. טעינת dataset מה-Hub
2. עיבוד הנתונים (preprocessing) עם `Dataset.map()`
3. חישוב מדדים (metrics)

אבל — זה רק "קצה הקרחון".

♦ מה הפרק הזה בא ללמד?

הפרק הזה נכנס לעומק של הספרייה  Datasets, ועונה על שאלות פרקטיות מאוד חשובות:

1. מה אם ה-dataset לא נמצא ב-Hub?

אין עובדים עם דאטה חיצוני (קבצים, JSON, CSV וכו') 

2. איך חותכים ומארגנים dataset?

Filtering, slicing, splitting 
ומה אם רוצים להשתמש ב-Pandas? 

3. מה עושים עם dataset גדול מדי?

כזה שיכול להפיל את ה-RAM 

4. מה זה בכלל:

- Memory Mapping
- Apache Arrow

 (זה קריטי לביצועים!)

5. איך יוצרים dataset משלך ומעלים אותו ל-Hub?

למה זה חשוב? 

כל הדברים האלה הם הבסיס ל: 

- Tokenization מתקדם
 - Fine-tuning אמיתי
 - עבודה עם דאטה גדול (Production-level)
-

 אינטואיציה (החלק הכי חשוב)

תחשבי על  Datasets כמו:

 "מנוע דאטה חכם"

במקום:

- לטעון הכל ל-RAM
- לעבוד ידנית עם Pandas

הספרייה נותנת לך:

- עבודה lazy (חכמה) עם דאטה
 - ביצועים גבוהים
 - יכולת לעבוד עם datasets ענקיים
-

 למה זה Game Changer?

כי ב-ML אמיתי:

 הדאטה שלך לא:

- קטן
- מסודר
- מוכן מראש

 אלא:

- גדול מאוד
 - מפוזר
 - צריך עיבוד
-

מה הולך להשתנות לך בראש 🔥

עד עכשיו:

"אני טוענת dataset ומאמנת מודל"

אחרי הפרק:

"אני מנהלת pipeline של דאטה בצורה יעילה וסקיילבילית"

סיכום בטבלה 📊

נושא	מה לומדים
טעינת דאטה	גם מחוץ ל-Hub
עיבוד	slicing, filtering, map
דאטה גדול	עבודה בלי לקרוס RAM
ביצועים	Arrow + memory mapping
יצירה	בניית dataset משלך

פרק 5.2

עד עכשיו:

למדת לטעון datasets מה-Hugging Face Hub 🙌

אבל בעולם האמיתי:

הדאטה שלך יהיה: 🙌

- על המחשב שלך
- בשרת
- בקבצים (CSV / JSON)

המטרה של הפרק 🔥

ללמוד איך לטעון dataset מכל מקור — לא רק מה-Hub

🧠 אינטואיציה (הכי חשוב)

תחשבי על זה ככה:

👉 Datasets הוא לא "מאגר datasets"
👉 הוא מנוע טעינה וניהול דאטה

דוגמה	טוען את הסקריפט	פורמט נתונים
<code>load_dataset("csv", data_files="my_file.csv")</code>	<code>csv</code>	CSV ו-TSV
<code>load_dataset("text", data_files="my_file.txt")</code>	<code>text</code>	קבצי טקסט
<code>load_dataset("json", data_files="my_file.jsonl")</code>	<code>json</code>	JSON וקווי JSON
<code>load_dataset("pandas", data_files="my_dataframe.pkl")</code>	<code>pandas</code>	מסגרות נתונים כבושות

💡 הדפוס הקבוע:

Python

```
load_dataset("type", data_files="path")
```

🚀 שלב 1: טעינת dataset מקומי
הורדת דאטה

Python

... wget!

👉 מוריד קובץ מהמחשב/אינטרנט

פתיחת קובץ zip

Python

... gzip -dkv!

טעינה

Python

```
from datasets import load_dataset

dataset = load_dataset(
    "json",
    data_files="file.json",
    field="data"
)
```

מה זה field? 💡

כשיש JSON מקונן (nested) 👉

JSON

```
{
  "data": [...]
}
```

צריך להגיד איפה הדאטה נמצא 👉

אינטואיציה 🧠

JSON לפעמים הוא "עטיפה" → field אומר איפה התוכן האמיתי

מה מקבלים? 📦

Python

```
DatasetDict({
  train: Dataset(...)
})
```

למה DatasetDict?

כי יש splits: 👉

- train
- test
- validation

🔥 שלב חשוב: עבודה עם splits

Python

```
data_files = {  
    "train": "...",  
    "test": "..."  
}
```

💡 אינטואיציה

את מגדירה איך לחלק את הדאטה

⚡ טריק חשוב: glob

Python

```
"data_files="*.json
```

👉 טוען כמה קבצים יחד

🔪 פיצ'ר חזק: טעינה מקובץ מכווץ

Python

```
"data_files="file.json.gz
```

👉 אין צורך לפתוח ידנית!

🧠 אינטואיציה

הספרייה "חכמה" — עושה preprocessing לבד

🌐 שלב 2: טעינת דאטה מרחוק (Remote)

Python

```
"...//:url = "https
```

```
        } = data_files
    , "...": train": url"
    "...": test": url"
    {

(load_dataset("json", data_files=data_files
```

אינטואיציה 💡

אין הבדל אמיתי בין: 🙌

- קובץ מקומי
- URL

מבחינת הקוד — זה אותו דבר 🙌

למה זה חשוב? 🔥

כי בעולם האמיתי:

הדאטה שלך כמעט אף פעם לא יהיה ב-Hub 🙌

תובנה עמוקה 🧠
לפני: 🔥

dataset = משהו שמורידים

אחרי: 🔥

dataset = משהו שאת יודעת לטעון מכל מקור

סיכום בטבלה 📊

מה זה גותן לך	יכולת
טעינה אחידה	load_dataset
שליטה במקור	data_files

field	עבודה עם JSON מורכב
splits	train/test
remote loading	עבודה עם שרתים
auto unzip	חיסכון בזמן

פרק 5.3

"Time to slice and dice"

👉 זה בעצם:

איך מנקים, מעבדים ומשנים dataset בצורה חכמה

🧠 אינטואיציה (הכי חשוב להבין)

תחשבי על זה ככה:

♦ יש לך dataset מהעולם האמיתי
→ הוא מלוכלך, לא אחיד, בעייתי

♦ המטרה:
👉 להפוך אותו ל-dataset שמתאים למודל

🔥 שלבי העבודה המרכזיים (ה-flow האמיתי)
1 טעינת dataset

Python

```
load_dataset("csv", data_files=..., delimiter="\t")
```

👉 גם CSV = TSV עם טאבים

2 לקחת sample קטן

Python

```
dataset.shuffle().select(range(1000))
```

למה? 💡

- להבין את הדאטה מהר
- לא לעבוד על כל הדאטה בהתחלה

3 ניקוי דאטה (Data Cleaning)

בעיות שמצאו:

- עמודה מיותרת (Unnamed: 0)
- אותיות גדולות/קטנות לא אחידות
- ערכים None
- HTML מוזר בטקסט

פתרונות:

שינוי שם עמודה ♦

Python

```
rename_column()
```

הסרת None ♦

Python

```
filter(lambda x: x["condition"] is not None)
```

נרמול (lowercase) ♦

Python

```
map()
```

MAP — הכלי הכי חשוב פה ⚡

Python

```
dataset.map(function)
```

זה כמו: 👉

"תריצי פונקציה על כל שורה"

אינטואיציה: 💡

זה כמו:

Python

```
for row in dataset:  
    process(row)
```

אבל:

מהיר ✓

מקבילי ✓

סקיילבילי ✓

יצירת פיצ'רים חדשים 🧱

Python

```
def compute_review_length(example):  
    return {"review_length":  
len(example["review"].split())}
```

מוסיף עמודה חדשה! 👉

למה זה חשוב?

ML = Feature Engineering 👉

סינון דאטה 🔍

Python

```
filter(lambda x: x["review_length"] > 30)
```

מסירים דאטה לא איכותי 👉

אינטואיציה: 💡

Garbage in → Garbage out

ניקוי טקסט (HTML) 🖌️

Python

```
html.unescape()
```

הופך: 👉

None

```
I&#039;m → I'm
```

הביצועים — החלק הקריטי 🚀
batched=True 💎

Python

```
map(..., batched=True)
```

עובד על הרבה שורות יחד 👉

למה זה מהיר? ⚡

- פחות קריאות פונקציה
- שימוש ב-vectorization
- מתאים ל-tokenizers מהירים

תוצאה אמיתית: 📊

זמן	מצב
59~ שניות	רגיל
10~ שניות	batched

פי 5-6 יותר מהר (!! 👉)

multiprocessing ⚙️

Python

```
map(..., num_proc=8)
```

👉 מפעיל כמה תהליכים במקביל

🧠 אינטואיציה:

- CPU = כמה "ידיים"
- num_proc = כמה ידיים עובדות

🔥 קטע חשוב מאוד — פיצול טקסטים (token overflow)

כשעושים:

Python

```
return_overflowing_tokens=True
```

👉 טקסט ארוך → מתפצל לכמה דוגמאות

! הבעיה:

datasetn גדל → mismatch בגודל

✅ פתרון 1 (פשוט):

Python

```
remove_columns=...
```

✅ פתרון 2 (חכם):

להשתמש ב-`overflow_to_sample_mapping`

👉 לשכפל את השדות המקוריים בהתאם

🔄 עבודה עם Pandas

Python

```
set_format("pandas")
```

👉 הופך ל-DataFrame

למה צריך את זה?

כי Pandas נותן:

- groupby
- סטטיסטיקות
- visualization

פיצול Train / Validation ✂️

Python

```
train_test_split()
```

חשוב כדי: 👉

- לא לעשות overfitting
- לא "לרמות" עם test

שמירת dataset 💾

Python

```
save_to_disk()  
to_csv()  
to_json()
```

Arrow = הכי מהיר 👉

סיכום על כל הפרק 📊

מה עושים	שלב
load_dataset	טעינה
sample	בדיקה
filter + map	ניקוי
map	פיצ'רים
batched + num_proc	שיפור ביצועים
map	tokenization
set_format	Pandas
train_test_split	split
save_to_disk	שמירה

Datasets זה לא רק "טעינת דאטה" 🤖👉

זה:

מערכת מלאה לניהול pipeline של דאטה בקנה מידה גדול 🔥

פרק 5.4

"Big data? 🤖 Datasets to the rescue"

השאלה המרכזית: 👉

מה עושים כשה-dataset שלך עצום (GB–TB)?

הבעיה האמיתית 💥

דוגמה:

- GPT-2 → אומן על 40GB טקסט
- The Pile → 825GB (!!)

👉 זה אומר:

❌ אי אפשר:

- לטעון ל-RAM
- לפעמים אפילו לא להוריד לדיסק

האינטואיציה הגדולה 🧠

😊 Datasets פותר את זה עם 2 רעיונות גאוניים:

1. Memory Mapping 🧩

👉 במקום לטעון את כל הדאטה לזיכרון:

📌 שומרים אותו בדיסק
📌 ורק "מצביעים" אליו מה-RAM

💡 אינטואיציה:

תחשבי על זה כמו:

- ספר בספרייה (הדיסק)
- את לא זוכרת הכל 🧠
- רק פותחת עמוד כשצריך 👉

🔥 התוצאה:

👉 dataset של ~20GB
👉 משתמש רק בכמה GB RAM

⚙️ איך זה עובד?

- Apache Arrow
- pyarrow

👉 פורמט מהיר במיוחד ל:

- קריאה
- עיבוד
- parallelization

ביצועים 🚀

הראו:

- מעבר על 64 ~ → 19GB שניות
- כלומר ~0.3GB/sec

👉 מטורף יחסית ל-Python

בעיה נוספת: גם הדיסק לא מספיק 🚀

אם יש לך:

- 825GB dataset

👉 גם דיסק לא מספיק 😊

הפתרון השני: Streaming 🚀

Python

```
load_dataset(..., streaming=True)
```

אינטואיציה: 🧠

👉 במקום:

להוריד הכל ואז לעבוד

👉 עושים:

“תביא לי שורה → תעבד → תביא הבאה”

IterableDataset: סוג חדש: 

כשעושים streaming:

אין indexing רגיל ✖
עובדים עם iterator ✔

Python

```
next(iter(dataset))
```

עיבוד תוך כדי תנועה ⚙️

Python

```
dataset.map(...)
```

עובד תוך כדי streaming 👉

גם פה יש ביצועים ⚡

Python

```
map(..., batched=True)
```

עדיין רלוונטי! 👉

shuffle ב-streaming 🔄

Python

```
shuffle(buffer_size=10000)
```

חשוב להבין: ⚠️

זה לא shuffle אמיתי על כל הדאטה 👉
רק בתוך buffer 👉

אינטואיציה:

• יש "חלון" של 10,000 דוגמאות

- shuffle רק בתוכו

split בלי להוריד דאטה ✂️

Python

```
train = dataset.skip(1000)
val = dataset.take(1000)
```

👉 בלי לשכפל data!

🔗 שילוב datasets (מאוד חשוב!)

Python

```
interleave_datasets([...])
```

🧠 אינטואיציה:

👉 לוקח:

- dataset A
- dataset B

👉 מחבר:

...A, B, A, B, A, B

למה זה שימושי?

👉 כשמאמנים LLM:

- רוצים דאטה מגוון
- ממקורות שונים

📊 דוגמה אמיתית

חיברו:

- PubMed (רפואי)
- FreeLaw (משפטי)

dataset אחד גדול 👉

לפני: 🚀

dataset = משהו שאני טוענת לזיכרון

אחרי: 🔥

dataset = stream של מידע שאני מעבדת תוך כדי

סיכום בטבלה 📊	
פתרון	בעיה
memory mapping	RAM קטן
streaming	דיסק קטן
IterableDataset	דאטה ענק
batched + Arrow	ביצועים
interleave	שילוב דאטה

פרק 5.5

מה הנושא? 📖

"Creating your own dataset"

הרעיון: 👉

לפעמים אין dataset → אז את בונה אותו בעצמך

אינטואיציה חזקה 🧠

עד עכשיו:

dataset = משהו מוכן

עכשיו:

pipeline = dataset של איסוף + ניקוי + העשרה

למה בכלל לבנות dataset? 🔥

דוגמה מהקטע:

Dataset של GitHub Issues → אפשר להשתמש בו ל:

- זיהוי bugs 🐛
 - (classification (bug / feature / question 🏷️
 - semantic search 🔍
-

ה-**FLOW** המלא (מאוד חשוב!) 🚀
1 איסוף נתונים (Data Collection)

Python

```
requests.get(url)
```

מושכים data מ-GitHub (API) 👉

אינטואיציה: 💡

dataset = API + scraping + aggregation

מה מקבלים?

JSON עם מלא מידע:

- title
- body
- labels
- user
- timestamps

⚠️ בעיה: rate limit

GitHub:

- בלי token → 60 requests/hour 🤖
- עם token → 5000/hour

פתרון:

Python

```
headers = {"Authorization": f"token {GITHUB_TOKEN}"}
```

🔄 הורדה מסיבית (pagination)

Python

```
page=1&per_page=100?
```

👉 לולאה על pages

💡 אינטואיציה:

API ≠ dataset
👉 צריך "לאסוף חתיכות" ולבנות dataset

📦 שמירה

Python

```
df.to_json(..., lines=True)
```

👉 JSONL = כל שורה = דוגמה

🔄 טעינה ל-HuggingFace

Python

```
load_dataset("json", ...)
```

שלב 2: ניקוי דאטה 🧹

בעיה:

GitHub מחזיר גם: 🙌

- Issues
- Pull Requests

זיהוי:

Python

```
pull_request is None
```

פתרון:

Python

```
map(lambda x: {"is_pull_request": ...})
```

אינטואיציה: 🧠

raw data תמיד מבולגן → צריך feature שיבדיל

שלב 3: העשרת הדאטה (Augmentation) 🔥

מוסיפים מידע חדש 🙌

דוגמה: comments

Python

```
get_comments(issue_number)
```

ואז:

Python

```
map(...)
```

מוסיף column חדש 🙌

אינטואיציה: 💡

dataset טוב =
data לא רק 🙌
אלא context (כמו comments) 🙌

שלב 4: העלאה ל-Hub 🚀

Python

```
dataset.push_to_hub("name")
```

עכשיו כל העולם יכול להשתמש בזה 🙌

שלב 5: Dataset Card 📄

README.md 🙌

כולל:

- מה הדאטה
- איך נאסף
- bias
- שימושים

אינטואיציה עמוקה 🧠

dataset ≠ רק קובץ 🙌

זה:

- data
- metadata
- documentation

סיכום כל התהליך 📊

מה עושים	שלב
API / scraping	איסוף
pagination + rate limit	ניהול
JSONL	שמירה
load_dataset	טעינה
map + filter	ניקוי
comments וכו'	העשרה
push_to_hub	שיתוף
dataset card	תיעוד

חיבור למה שלמדת קודם 💡

איך זה מתחבר	פרק
ניקוי	slice & dice
עבודה בסקייל	big data
יצירה מאפס	פה

מה הנושא? 📖
"Semantic search with FAISS"

המטרה: 📌

לבנות מנוע חיפוש שלא מחפש מילים — אלא משמעות

אינטואיציה (החלק הכי חשוב!) 🧠
חיפוש רגיל (Keyword search) 🔍

אם תחפשי:

"load dataset offline"

הוא יחפש: 📌

• מילים זהות בלבד

✗ בעיה:

- לא מבין משמעות
- מפספס ניסוחים שונים

Semantic Search 🤖

מחפש לפי משמעות 📌

לדוגמה:

- "offline dataset"
- "load data without internet"

📌 יביא תוצאות דומות!

איך זה עובד? 🔥
שלב 1: הופכים טקסט לוקטור (Embedding)

None

טקסט → וקטור (768 מספרים)

אינטואיציה: 💡

תחשבי:

- כל משפט = נקודה במרחב
- משפטים דומים = קרובים

שילבי הבנייה (Pipeline מלא) 🧩

1 טעינת dataset

Python

```
load_dataset("github-issues")
```

2 ניקוי דאטה

Python

```
filter(...)
```

מסירים: 🙅

- Pull requests
- בלי comments

3 שמירת עמודות חשובות

Python

```
["title", "body", "comments", "html_url"]
```

4 explode comments

כל comment = שורה 🙅

למה? 💡
כי:

anחנו מחפשים תשובות → comments

5 סינון comments קצרים

Python

```
filter(lambda x: len(...) > 15)
```

6 יצירת טקסט אחד

Python

```
text = title + body + comment
```

אינטואיציה: 💡

נותנים למודל context מלא

שלב קריטי: Embeddings 🚀
מודל:

Python

```
sentence-transformers/multi-qa-mpnet-base-dot-v1
```

הפיכה לוקטור:

Python

```
embedding = model(text)
```

מקבלים: 👉

Python

```
[numbers 768]
```

:pooling

Python

```
CLS token
```

מייצג את כל המשפט 👉

אינטואיציה: 🧠

לוקחים את "המשמעות הכללית" של המשפט

יצירת embeddings לכל הדאטה ⚡

Python

```
dataset.map(...)
```

FAISS: שלב הבא: 🔍
מה זה?

מנוע חיפוש מהיר לוקטורים 🙌

יצירת index:

Python

```
add_faiss_index("embeddings")
```

אינטואיציה: 💡

במקום:

• לעבור על כל הדאטה

FAISS עושה: 🙌

“מצא לי הכי קרובים”

חיפוש בפועל 🔍
1 הופכים שאלה לוקטור

Python

```
question_embedding = get_embeddings(question)
```

2 מחפשים דמיון

Python

```
get_nearest_examples(...)
```

3 מקבלים:

- scores (כמה דומה)
- results (התשובות)

דוגמה אמיתית 📊

שאלה:

"How can I load a dataset offline?"

👉 המערכת מצאה תשובות רלוונטיות 🔥

תובנה עמוקה 🧠
למה זה עובד טוב? 🔥

כי:

- לא תלוי במילים
- מבין meaning

שיטה	איך עובד	בעיה
keyword	מילים	לא מבין משמעות
semantic	embeddings	חכם

משפט מפתח 🧠

"Search is no longer about words — it's about meaning"

החיבור הגדול 🔥

מה שעשית עכשיו = הבסיס ל:

- ChatGPT retrieval 🤖
- RAG systems 🔍
- QA systems 📖
- LLM pipelines 🧠

סיכום pipeline 📊

שלב	פעולה
ניקוי	filter
reshape	explode
הכנה	concatenate
embedding	model
index	FAISS
query	embedding
search	nearest neighbors

מה זה חיפוש אסימטרי (Asymmetric Semantic Search)? 🧠

זה מצב שבו: 👉

! הצדדים של החיפוש לא באותו סוג או אורך

הדוגמה הכי קלאסית 🔥
יש לך:

שאלה קצרה: 🔍

None

How to load dataset offline

מול: 📄

None

תגובה ארוכה / מסמך / **issue**

👉 הם לא "שווים" → אסימטרי

♦ לעומת חיפוש סימטרי
סימטרי:

None

משפט ↔ משפט

למשל:

• דמיון בין שני משפטים

אסימטרי:

None

שאלה ↔ מסמך

🔥 למה זה חשוב?

כי המודל צריך:

👉 להבין:

- מה השאלה מחפשת
- איך זה מופיע בתוך טקסט ארוך

🧠 איך פותרים את זה?

משתמשים במודלים מיוחדים, למשל:

None

`sentence-transformers/multi-qa-mpnet-base-dot-v1`

👉 שמותאמים ל-QA (שאלה-תשובה)

◆ מה עושים בפועל?

1. ממירים שאלה ל-embedding
2. ממירים מסמכים ל-embedding
3. מחשבים similarity
4. מחזירים הכי דומים

🧠 אינטואיציה

תחשבי:

👉 את שואלת שאלה קצרה
👉 ומחפשת תשובה בתוך מאמרים

זה לא:

- משפט מול משפט

זה:

- שאלה מול עולם של טקסט

פרק 5.7

👉 זה סיכום של כל מה שלמדנו בפרק 5
על ספריית Datasets 😊

🧠 התמונה הגדולה

אם אני צריך לסכם במשפט אחד:

למדנו איך לקחת דאטה גולמי → ולהפוך אותו ל-pipeline מוכן למודל

🔥 מה את יודעת עכשיו (פירוק אמיתי)
1️⃣ טעינת דאטה מכל מקום

👉 לא רק מה-Hub:

- מקומי (CSV / JSON)
- שרתים
- API
- streaming

💡 אינטואיציה

data source לא משנה — הכל נכנס לאותה מערכת

2️⃣ עיבוד דאטה (הכי חשוב!)

Python

```
map()  
filter()
```

👉 זה הלב של העבודה

🧠 מה זה באמת נותן לך?

- ניקוי
- טרנספורמציה
- feature engineering

3 עבודה עם פורמטים שונים

Python

```
set_format("pandas")
```

👉 מעבר בין:

- Dataset
- Pandas
- NumPy

💡 אינטואיציה

😊 Datasets = שכבת abstraction מעל הכל

4 יצירת dataset משלך

👉 כולל:

- איסוף (API)
- ניקוי
- העשרה
- שמירה
- שיתוף

🔥 זה כבר level של פרויקט אמיתי

5 עבודה עם דאטה גדול

👉 בלי לקרוס:

- memory mapping
- streaming

🧠 תובנה

את לא טוענת דאטה — את "זורמת" דרכו

6 Embeddings + Semantic Search

👉 בנית:

- embeddings

- FAISS index
- מנוע חיפוש חכם

זה כבר LLM-level systems 🌟

סיכום בטבלה 📊

מה זה נותן לך	יכולת
גמישות	load_dataset
שליטה בדאטה	map/filter
עבודה עם כלים	set_format
סקייל	streaming
פרויקטים אמיתיים	dataset creation
חיפוש חכם	embeddings + FAISS

ה-Takeaway הכי חשוב 🧠

- 👉 הפרק הזה לא על ספרייה
 - 👉 הוא על איך עובדים עם דאטה בעולם האמיתי
-

מה הולך לקרות בפרק הבא? 🚀

Chapter 7:

👉 NLP tasks אמיתיים:

- classification
- NER
- QA
- וכו'

💡 חיבור למה שעשית

כל מה שלמדת עכשיו = הכנה ל:

- Fine-tuning
- Training pipelines
- Production ML

פרק 5.8

שאלה 1

The `load_dataset()` function allows you to load a dataset from which of the following locations?
מאילו מקומות אפשר לטעון dataset באמצעות `load_dataset()`?

- A.
Locally, e.g. on your laptop
מקומי (על המחשב שלך)
- B.
The Hugging Face Hub
ה-Hub של Hugging Face
- C.
A remote server
שרת מרוחק

שאלה 2

Which command produces a random sample of 50 elements from dataset?
איזה קוד יוצר דגימה אקראית של 50 איברים?

- A.
`dataset.sample(50)`
- B.
`dataset.shuffle().select(range(50))`

C.
`dataset.select(range(50)).shuffle()`

שאלה 3

?"Which approach filters pets whose names start with "L"
איזו גישה מסננת חיות ששמן מתחיל ב-L?

A.
`pets_dataset.filter(lambda x: x['name'].startswith('L'))`

B.
`pets_dataset.filter(lambda x['name'].startswith('L'))`

C.
Function `filter_names + filter`

שאלה 4

?What is memory mapping
מה זה memory mapping?

A.
Mapping between CPU and GPU RAM
מיפוי בין CPU ל-GPU

B.
Mapping between RAM and filesystem storage
מיפוי בין זיכרון לדיסק

C.
Mapping between two cache files
מיפוי בין קבצים

שאלה 5

?What are the main benefits of memory mapping
מה היתרונות של memory mapping?

A.
Faster than disk read/write
מהיר יותר מקריאה רגילה

B.
Access large files without loading all into RAM
גישה לקבצים גדולים בלי לטעון הכל

C.
Consumes less battery
חוסך סוללה

שאלה 6

?Why does this code fail
למה הקוד נכשל?

```
from datasets import load_dataset
```

```
dataset = load_dataset("allocine", streaming=True, split="train")
```

```
dataset[0]
```

A.
Dataset too large
הדאטה גדול מדי

B.
Tries to access an IterableDataset
מנסה לגשת ל-IterableDataset

C.
No train split
אין train

שאלה 7

?What are the benefits of creating a dataset card
מה היתרונות של dataset card?

A.
Explains intended use
מסביר שימוש

B.
Highlights bias
מציג הטיות

C.
Helps others use dataset
עוזר לאחרים להשתמש

שאלה 8

?What is semantic search
?מה זה semantic search

A.
Exact keyword matching
חיפוש לפי מילים

B.
Understanding contextual meaning
הבנת משמעות

C.
Improving accuracy
שיפור דיוק

שאלה 9

:For asymmetric semantic search, you usually have
:בחיפוש אסימטרי יש בדרך כלל:

A.
Short query + long document
שאלתה קצרה וטקסט ארוך

B.
Same length
אותו אורך

C.
Long query + short answer
שאלתה ארוכה

שאלה 10

Can 🤗 Datasets be used for other domains like speech
האם אפשר להשתמש ב-Datasets גם לתחומים כמו speech?

A.
No
לא

B.
Yes
כן

שאלה 1

תשובות נכונות: A, B, C ✓

הסבר: 💡
אפשר לטעון dataset מכל מקור — מקומי, Hub או שרת מרוחק.

שאלה 2

תשובה נכונה: B ✓

Python

```
((dataset.shuffle()).select(range(50
```

הסבר: 💡
shuffle → ערבוב
select → בחירה
ככה מקבלים דגימה אקראית אמיתית.

שאלה 3

תשובות נכונות: A, C ✓

הסבר: 💡
גם lambda תקין וגם פונקציה חיצונית עובדים.
אפשרות B שגויה בגלל תחביר לא נכון.

שאלה 4

תשובה נכונה: B ✓

הסבר: 💡
Memory mapping = חיבור בין RAM לדיסק כדי לא לטעון הכל לזיכרון.

שאלה 5

תשובות נכונות: A, B ✓

הסבר: 💡

- מאפשר עבודה מהירה יותר
- מאפשר עבודה עם דאטה גדול מאוד

C לא נכון. ✗

שאלה 6

תשובה נכונה: B ✓

הסבר: 💡
Streaming IterableDataset → indexing (`dataset[0]`).

שאלה 7

תשובות נכונות: A, B, C ✓

הסבר: 💡
Dataset card

- מסביר שימוש
- מציג bias
- עוזר לאחרים להשתמש

שאלה 8

✓ תשובה נכונה: B

💡 הסבר:

Semantic search = חיפוש לפי משמעות, לא לפי מילים.

שאלה 9

✓ תשובה נכונה: A

💡 הסבר:

שאלתה קצרה → תשובה ארוכה (כמו QA או ChatGPT).

שאלה 10

✓ תשובה נכונה: B

💡 הסבר:

הספרייה כללית — לא רק NLP (גם speech וכו').

פרק 6.1

בפרק 3:

👉 השתמשת ב-tokenizer קיים (מוכן מראש)

אבל עכשיו שואלים:

? מה עושים אם רוצים לאמן מודל מאפס?

🔥 התשובה:

צריך גם:

👉 לאמן tokenizer מאפס

🧠 למה זה בכלל חשוב?

כי tokenizer תלוי ב-data

💡 דוגמה מהטקסט:

👉 tokenizer באנגלית
❌ לא יעבוד טוב על יפנית

למה?

- באנגלית יש רווחים
- ביפנית אין

🧠 אינטואיציה חזקה

tokenizer = הדרך שבה המודל "קורא" טקסט

אם הקריאה לא טובה →
👉 המודל לא יבין

מה הולכים ללמוד בפרק הזה? 🔥

1 איך לאמן tokenizer חדש

👉 על dataset שלך

2 Fast Tokenizers

👉 tokenizer מהיר (Rust-based)

3 אלגוריתמים שונים

- BPE
- WordPiece
- Unigram

4 איך לבנות tokenizer מאפס

👉 שלב אחרי שלב

מה זה בעצם "לאמן tokenizer"?

👉 למצוא:

- איך לפצל מילים
- אילו תתי-מילים להשתמש

💡 אינטואיציה:

במקום:

None

["hello → ["hello

לפעמים עושים:

None

`["hello" → ["he", "llo`

למה עושים את זה? 🧠

כדי:

- להתמודד עם מילים חדשות
- לצמצם vocabulary
- לשפר generalization

Fast vs Slow Tokenizers 🔥

הסבר

Python

Rust
(מהיר
מאוד)

תובנה חשובה 🧠

tokenizer זה לא preprocessing קטן
זה חלק קריטי מהמודל

למה זה חשוב לפרקים הבאים? 🚀

בפרק 7: 📌

תבני מודל שפה (למשל לקוד Python)

זוה דורש: 📌

tokenizer מותאם

סיכום 📊

נושא	מה לומדים
tokenizer	איך מפצלים טקסט
training	בנייה מאפס
algorithms	BPE / WordPiece / Unigram
fast tokenizers	ביצועים
שימוש	הכנה ל-LLM

takeaway 🧠

אם הדאטה שלך מיוחד → tokenizer חייב להיות מותאם

פרק 6.2

אם:

אין מודל לשפה שלך 🙌

או

הדאטה שלך שונה מאוד מהדאטה המקורי 🙌

אז:

! צריך לאמן tokenizer חדש

אבל רגע — מה זה "לאמן tokenizer"?

זה לא כמו אימון מודל! 🙌

⚠️ הבדל קריטי

Tokenizer Training

Model Training

סטטיסטי

Gradient Descent

דטרמיניסטי

רנדומלי

תמיד אותו פלט

תלוי seed

אינטואיציה 💡

tokenizer לא "לומד" כמו מודל
הוא פשוט מוצא את ה-subwords הכי טובים

🔥 שלב 1: בניית corpus

👉 צריך הרבה טקסטים

בדוגמה:

- קוד Python
- dataset: CodeSearchNet

Python

```
from datasets import load_dataset
```

```
("raw_datasets = load_dataset("code_search_net", "python
```

🧠 למה קוד?

כי:

👉 זה "שפה" שונה מאנגלית רגילה

♦ איזה עמודה משתמשים?

Python

```
whole_func_string
```

👉 הפונקציה כטקסט

🚀 שלב 2: להכין iterator

❌ לא עושים:

Python

```
(...)list
```

👉 כי זה יקרוס בזיכרון

עושים generator ✓

Python

```
def get_training_corpus():
    for i in range(0, len(dataset), 1000):
        yield dataset[i:i+1000]
```

אינטואיציה 🧠

טוענים דאטה "בזרימה" (streaming-like)

שלב 3: טעינת tokenizer בסיס 🔥

Python

```
old_tokenizer = AutoTokenizer.from_pretrained("gpt2
```

למה לא מאפס? 💡

כי:

כבר יש: 👉

- אלגוריתם
- special tokens

רק ה-vocabulary משתנה 👉

דוגמה לבעיה 💎

Tokenizer של GPT2 על קוד:

None

```
['add_numbers' → ['add', '_', 'n', 'umbers
```

❌ לא טוב לקוד

🚀 שלב 4: אימון tokenizer חדש

Python

```
)tokenizer = old_tokenizer.train_new_from_iterator  
    , training_corpus  
    52000  
    (  
    )
```

💡 מה זה 52000?

👉 גודל ה-vocabulary

🧠 אינטואיציה

tokenizer לומד:
איזה subwords הכי נפוצים בדאטה שלך

🔥 תוצאה

עכשיו tokenizer יודע:

✓ לזהות indentation
✓ לזהות ""
✓ לפצל נכון לפי _

דוגמה

לפני:

None

tokens 36

אחרי:

None

tokens 27

יותר יעיל! 👉

למה זה חשוב? 🧠

כי:

פחות tokens = 👉

- פחות חישוב
- הבנה טובה יותר

דוגמאות חכמות 💡

Tokenizer חדש מזהה:

- `ĈĜĜĜ` → indentation
- `Ġ" " "` → docstring
- `[LinearLayer` → `["Linear", "Layer`

שלב 5: שמירה 🚀

Python

```
("tokenizer.save_pretrained("my-tokenizer
```

◆ שלב 6: העלאה ל-Hub

Python

```
("tokenizer.push_to_hub("my-tokenizer
```

🧠 אינטואיציה

tokenizer הוא asset כמו מודל

סיכום

מה עושים	שלב
אוספים טקסט	dataset
חוסכים זיכרון	generator
התחלה חכמה	base tokenizer
בונים vocab	train
שומרים	save
משתפים	push

🧠 takeaway הכי חשוב

tokenizer חייב להתאים לדאטה שלך

פרק 6.3

עד עכשיו:

👉 השתמשת ב-tokenizer רק בשביל:

- tokenize (טקסט → tokens)
- decode (tokens → טקסט)

אבל עכשיו:

🔥 tokenizer (ובעיקר fast tokenizer) יודע לעשות הרבה יותר

🧠 הרעיון המרכזי

👉 Fast tokenizer = לא רק מפרק טקסט

👉 הוא גם שומר מידע על איפה כל token הגיע בטקסט המקורי

💡 חשוב:

Fast vs Slow ⚡	
סוג	הסבר
slow	כתוב ב-Python
fast	כתוב ב-Rust (מהיר מאוד)
💡 חשוב:	

- על טקסט קטן → לא תמיד רואים הבדל
- על הרבה דאטה → fast הרבה יותר מהיר

📦 מה מקבלים מה-tokenizer?

👉 לא dict רגיל

👉 אלא:

Python

BatchEncoding

זה dict + יכולות נוספות 💡

Offset Mapping: היכולת הכי חשובה: 🔥

לכל token יש מיקום בטקסט המקורי 🙌

לדוגמה:

None

```
['Sylvain' → ['S', '##yl', '##va', '##in
```

אבל fast tokenizer יודע: 🙌

- איפה מתחיל כל חלק
- לאיזה מילה הוא שייך

למה זה חשוב? 🧠

כי זה מאפשר:

לחבר חזרה tokens → מילים 🙌
לעבוד עם tasks כמו: 🙌

- NER
- QA

יכולות מרכזיות 💎

tokens[1]()

Python

encoding.tokens()

👉 מחזיר tokens ישירות (בלי decode)

`word_ids()`

👉 אומר לאיזה מילה כל token שייך

Python

```
encoding.word_ids()
```

💡 אינטואיציה:

כמה tokens → מילה אחת

`mapping` בין טקסט ל-tokens ³

Python

```
word_to_chars()
```

```
token_to_chars()
```

👉 ממפה:

- token → תווים בטקסט
- מילה → מיקום בטקסט

🔥 למה זה משנה?

כי:

👉 מודל עובד על tokens

👉 אבל אנחנו רוצים תשובה על טקסט

🧩 דוגמה (NER)

מודל מחזיר:

None

`['S', '##yl', '##va', '##in']`

אבל אנחנו רוצים:

None

`"Sylvain"`

👉 offset mapping פותר את זה

🚀 איך pipeline עובד מאחורי הקלעים?

שלבים:

1. Tokenization
2. Model → logits
3. Post-processing

◆ חיזוי (Prediction)

המודל מחזיר:

Python

`logits → probabilities → labels`

labels 💡

None

`B-PER → התחלה של אדם`

`I-PER → המשך של אדם`

`0 → לא ישות`

אינטואיציה 🧠

כל token מקבל label
ואז צריך לחבר אותם חזרה למילים

איך מחברים tokens לישות אחת? 🔥

לדוגמה:

None

Hu + ##gging + Face

הופך ל: 👉

None

Hugging Face

איך?

👉 לפי offsets (מיקום בטקסט)

רעיון חזק ⚡

במקום לשחק עם ##
פשוט לוקחים substring מהטקסט המקורי

למה זה גאוני? 🧠

כי זה עובד לכל tokenizer:

- BERT
 - GPT
 - SentencePiece
-

takeaway הכי חשוב 🧠

tokenizer טוב = חיבור בין מודל לטקסט אמיתי

סיכום החלק 🔥

יכולת	למה זה חשוב
fast tokenizer	ביצועים
BatchEncoding	מידע נוסף
offset mapping	חיבור לטקסט
word_ids	הבנת מילים
post-processing	NER / QA

פרק 6.4

👉 איך משתמשים ב-tokenizer (ובעיקר offsets) כדי למצוא תשובה בתוך טקסט

הרעיון המרכזי 🧠

ב-Question Answering:

המודל לא מחזיר טקסט
👉 הוא מחזיר מיקומים (indices)

אינטואיציה 💡

במקום:

"התשובה היא: ..."

המודל אומר:

"התשובה מתחילה בטוקן 21 ונגמרת ב-24"

שלב 1: שימוש ב-pipeline 🚀

Python

```
("pipeline("question-answering
```

מקבלים: 🙌

Python

```
}  
  
,"..." : "answer"  
  
,"start": 78"  
  
"end": 105"  
  
{
```

חשוב 💡

🙌 pipeline כבר עושה הכל מאחורי הקלעים

שלב 2: איך זה עובד בפנים? 🔥

Tokenization📄

Python

```
(tokenizer(question, context
```

חשוב: 🙌

- question קודם
- context אחריו

מודל 2

Python

```
(outputs = model(**inputs
```

המודל מחזיר: 🙌

- start_logits
- end_logits

אינטואיציה 🧠

לכל token: 🙌

- כמה סיכוי להיות התחלה
- כמה סיכוי להיות סוף

שלב 3: הפיכת logits ל-probabilities 🔥

Python

```
()softmax
```

שלב קריטי: masking ⚠️

צריך להתעלם מ: 🙌

- question
- padding

Python

```
start_logits[mask] = -10000
```

למה? 🧠

שלא נבחר תשובה מהשאלה 😊

שלב 4: מציאת span 🚀

מחשבים: 🙌

Python

```
start_prob * end_prob
```

לכל זוג אפשרי

תנאי חשוב 💡

$start \leq end$ 🙌

איך עושים את זה?

Python

```
[ : , scores = start_probs[ : , None ] * end_probs[ None  
( scores = torch.triu(scores
```

שלב 5: בחירת התשובה 🔥

Python

```
argmax
```

מוצאים: 🙌

- start_index
- end_index

שלב 6: מעבר ל-text 🚀

פה נכנס הקסם ✨

משתמשים ב-offset mapping 🙌

Python

```
[start_char, _ = offsets[start_index  
    [end_char = offsets[end_index , _  
  
[answer = context[start_char:end_char
```

אינטואיציה 🧠

tokens → חוזרים לטקסט המקורי

בעיה אמיתית: טקסט ארוך 🔥

אם הטקסט ארוך:

נחתך (truncation) ❌
והתשובה יכולה להיעלם 🙌

הפתרון: split + overlap 🚀

Python

```
return_overflowing_tokens=True  
  
stride=128
```

אינטואיציה 💡

מחלקים את הטקסט ל-chunks עם חפיפה ביניהם

דוגמה:

None

chunk1: tokens 0-384

chunk2: tokens 256-640

מה מקבלים?

Python

overflow_to_sample_mapping

אומר מאיפה הגיע כל chunk

שלב 7: הרצה על כל chunk

לכל chunk:

- מחשבים answer
- שומרים score

Python

```
[... , (candidates = [(start, end, score
```

שלב 8: בחירת התשובה הכי טובה

בוחרים score הכי גבוה

המודל "מחפש תשובה" בכל חלק
ואז בוחר את הכי בטוח

takeaway הכי חשוב 🧠

— QA model לא מחזיר תשובה
הוא מחזיר מיקום של תשובה

סיכום pipeline 📊

פעולה	שלב
question + context	tokenize
start/end logits	model
הסרת רעש	masking
probabilities	softmax
חישוב כל האפשרויות	span
חזרה לטקסט	offset
טיפול בטקסט ארוך	split
בחירת best answer	select

פרק 6.5

לפני שעושים tokenization אמיתי:

יש 2 שלבים מקדימים: 🙌

1. Normalization
2. Pre-tokenization

התמונה הגדולה 🧠

None

Raw Text → Normalization → Pre-tokenization →
Tokenization

שלב 1: Normalization 🔥
מה זה? 📖

ניקוי כללי של הטקסט 🙌

דוגמאות:

- lowercase
- הסרת accents
- תיקון Unicode
- הסרת רווחים מיותרים

דוגמה: 💡

None

"?Héllò hōw are ü"

הופך ל: 🙌

None

"?hello how are u"

אינטואיציה 🧠

עושים את הטקסט "אחיד" לפני שמפרקים אותו

חשוב !

Tokenizer שונה → normalization שונה

:BERT uncased

מוריד אותיות גדולות ✓
מוריד accents ✓

:BERT cased

לא משנה אותיות ✗
שומר accents ✗

Pre-tokenization שלב 2: 🔥
מה זה? 📘

פירוק ראשוני למילים 👉

דוגמה:

None

"?Hello, how are you"

הופך ל: 👉

None

["?", "Hello", ",", "how", "are", "you"]

חשוב: 💡

tokenizer כבר: 🙌

- מזהה מילים
- שומר offsets (מיקום בטקסט)

אינטואיציה 🧠

זה כמו "חיתוך גס" לפני חיתוך מדויק

הבדל בין tokenizers 🔥

BERT ♦

- מפריד לפי:
 - רווחים
 - סימני פיסוק
- 🙌 מתעלם מרווחים כפולים

GPT-2 ♦

- שומר רווחים (!)
- משתמש ב- $\hat{G}$

None

"how" → " $\hat{G}$ how "

(T5 (SentencePiece ♦

- משתמש ב- $_$ לרווחים
 - לא מפריד לפי פיסוק
-

למה זה חשוב? 🧠

כי:

👉 כל tokenizer "רואה" את הטקסט אחרת

🔥 SentencePiece (מאוד חשוב!)
📘 מה זה?

👉 אלגוריתם שעובד על תווים (Unicode)

💡 תכונות:

- לא צריך pre-tokenization
 - מתאים לשפות בלי רווחים (יפנית, סינית)
 - reversible
-

🧠 אינטואיציה

לא מניח שיש מילים — לומד הכל מהדאטה

🔥 מה זה reversible?

👉 אפשר לשחזר טקסט מהטוקנים

דוגמה:

None

Hello _world_

👉 חוזר ל:

None

Hello world

חשוב ⚠️

BERT ✖️ לא reversible
(כי הוא משנה רווחים)

סקירה של האלגוריתמים 🔥	
אלגוריתם	רעיון
BPE	מחבר זוגות תווים נפוצים
WordPiece	בוחר לפי score
Unigram	מתחיל גדול ומסיר
אינטואיציה פשוטה 🧠	
שיטה	איך חושבת
BPE	מה הכי נפוץ?
WordPiece	מה הכי חכם סטטיסטית?
Unigram	מה אפשר למחוק בלי לפגוע?
סיכום 📊	
שלב	מה קורה
Normalization	ניקוי טקסט
Pre-tokenization	פירוק למילים
Tokenization	subwords ⬇️

takeaway 🧠 הכי חשוב

tokenizer לא מתחיל מ-tokenization
הוא מתחיל מניקוי והכנה

פרק 6.6

מה זה BPE? ♦

אלגוריתם לפירוק מילים ל-subwords 👉

במקור:

נועד לדחיסת טקסט 👉

ואז:

שימש ל-GPT ו-GPT-2 👉

אינטואיציה הכי חשובה 🧠

מתחילים מאותיות

ומחברים אותן בהדרגה לתת-מילים נפוצות

הרעיון המרכזי 🔥

כל פעם: 👉

מחפשים את הזוג הכי נפוץ של תווים/טוקנים

ומחברים אותו 👉

שלב 1: Vocabulary בסיסי 🚀

נניח שיש מילים:

None

hug, pug, pun, bun, hugs

ה-vocab הראשוני: 👉

None

[b, g, h, n, p, s, u]

למה אותיות? 🧠

כי:

זה הבסיס הכי קטן 🙌

שלב 2: סופרים תדירויות 🔥

לדוגמה:

None

(hug (10

(pug (5

(pun (12

שלב 3: מחפשים זוג הכי נפוץ 🚀

למשל: 🙌

None

$(u, "g") \rightarrow 20$ פעמים

Merge ראשון 🔥

None

$(u, "g") \rightarrow "ug"$

אחרי merge:

None

"h ug"

"p ug"

אינטואיציה 🧠

המודל "לומד" ש-"ug" זה יחידה חשובה

חוזרים שוב 🔄

Merge הבא:

None

"u", "n") → "un")

Merge הבא:

None

"h", "ug") → "hug")

מה קורה לאורך זמן? 🔥

tokens נהיים: 👉

- יותר ארוכים
- יותר "משמעותיים"

אינטואיציה 🧠

שלב	token
התחלה	אותיות
אמצע	תתי מילים
סוף	מילים שלמות

איך tokenization עובד בפועל? 🚀

שלבים:

1. Normalization
2. Pre-tokenization
3. פירוק לאותיות
4. הפעלת merge rules

דוגמה 💡

עם rules:

None

$u, "g" \rightarrow ug$
 $h, "ug" \rightarrow hug$

המילה:

None

$["bug \rightarrow ["b", "ug$

המילה:

None

`["thug" → ["UNK", "hug`

בעיה ⚠️

אם אות לא קיימת: 🙅

מקבלים [UNK] ❌

פתרון בעולם האמיתי 🔥

GPT-2 משתמש ב-byte-level BPE 🙅

מה זה? 💡

עובד על 256 bytes (תווים) 🙅

אין UNK ✓

תומך בכל תו (גם emoji) ✓

אינטואיציה 🧠

במקום אותיות → עובדים על bytes

מה האלגוריתם לומד? 🔥

- merge rules
- vocabulary

סיכום תהליך training

שלב	פעולה
init	אותיות
count	תדירויות
merge	זוג הכי נפוץ
repeat	עד vocab size

Implementation (הרעיון)

1. סופרים מילים
2. מפצלים לאותיות
3. מחשבים זוגות
4. עושים merge
5. חוזרים

takeaway הכי חשוב

BPE לומד "לבנות מילים" מהתדירות שלהן

חיבור למה שלמדת

נושא	קשר
tokenizer training	פה זה קורה
vocab	נבנה כאן
subwords	התוצאה

פרק 6.7

מה זה WordPiece? ♦

האלגוריתם של BERT 🙌

דומה ל-BPE אבל עם הבדל חשוב מאוד:

! לא בוחר לפי תדירות — אלא לפי score חכם

רעיון מרכזי 🔥

רעיון מרכזי 🔥

במקום לבחור זוג הכי נפוץ:

מחשבים: 🙌

$$score = \frac{freq(pair)}{freq(first) \times freq(second)}$$

אינטואיציה הכי חשובה 🧠

אלגוריתם

איך מחליט merge

BPE

הכי נפוץ

WordPiece

הכי "משתלם" סטטיסטית

WordPiece מעדיף זוגות שהם:

- מופיעים יחד הרבה
- אבל בנפרד לא נפוצים

♦ התחלה (כמו BPE)

מילים:

None

hug, pug, pun, bun, hugs

🚀 שלב 1: פיצול עם prefix

None

word" → w ##o ##r ##d"

💡 למה ## ?

👉 אומר:

"זה המשך של מילה"

🔥 vocabulary ראשוני

None

`["b", "h", "p", "##g", "##n", "##s", "##u"]`

שלב 2: חישוב score 🚀

לדוגמה: 👉

• ("u", "##g##") מופיע הרבה
✗ אבל גם "##u" לבד מאוד נפוץ → score נמוך

merge ראשון 🔥

None

`"g", "##s") → "##gs##")`

למה? 🧠

כי:

👉 החלקים עצמם פחות נפוצים
👉 ולכן score גבוה יותר

ממשיכים כמו BPE 🔄

אבל תמיד לפי score, לא frequency

🔥 הבדל קריטי נוסף
! WordPiece שומר רק vocabulary

👉 לא שומר merge rules

איך tokenization עובד? 🚀

אלגוריתם:

לכל מילה: 📌

1. מוצאים את ה-subword הכי ארוך שקיים
2. חותכים
3. ממשיכים

דוגמה 💡

מילה:

None

hugs

:WordPiece

None

["hug", "##s"]

:BPE

None

["hu", "##gs"]

אינטואיציה 🧠

WordPiece תמיד מחפש את החתיכה הכי גדולה שאפשר

דוגמה נוספת 🔥

None

bugs

שלבים:

- ✓ 1. "b"
- ✓ 2. "u##"
- ✓ 3. "gs##"

תוצאה: 👉

None

["b", "##u", "##gs"]

⚠️ הבדל חשוב מאוד מ-BPE
אם משהו לא קיים:

👉 WordPiece:

None

["[UNK]"]

💡 דוגמה

None

["[mug → ["[UNK

BPE היה עושה:

None

["UNK", "ug"]

אינטואיציה 🧠

אלגוריתם	התנהגות
BPE	חלקי UNK
WordPiece	הכל או כלום

סיכום training 🔥

שלב	פעולה
split	עם ##
score	לפי נוסחה
merge	הכי משתלם
repeat	עד vocab size

סיכום tokenization 🚀

שלב	פעולה
word	מתחילים מהמילה
longest match	הכי ארוך
split	חותכים
repeat	עד סוף

השוואה סופר חשובה 📊

תכונה	BPE	WordPiece
בחירה	frequency	score
merge rules	נשמרים	לא
tokenization	לפי merges	longest match
UNK	חלקי	מלא

פרק 6.8

מה זה Unigram? ♦

אלגוריתם טוקניזציה מתקדם
בשימוש עם SentencePiece 👉

מודלים שמשתמשים בו:

- T5
- XLNet
- mBART

האינטואיציה הכי חשובה 🧠	
אלגוריתם	איך עובד
BPE	בונה מלמטה למעלה
WordPiece	בונה חכם
Unigram 🔥	מתחיל גדול ומסיר

רעיון מרכזי 🔥

מתחילים עם vocabulary גדול מאוד
 ואז מסירים tokens "מיותרים" 🙅

אינטואיציה 🧠

→ במקום לבנות tokens
 מוחקים את מה שלא חשוב

שלב 1: יצירת vocab גדול 🚀

דוגמה:

None

[..., "h", "u", "g", "hu", "ug", "hug", "ugs"]

שלב 2: מחשבים הסתברויות 🔥

👉 לכל token יש probability:

$$P(\text{token}) = \frac{\text{frequency}}{\text{total}}$$

🧠 אינטואיציה

כמה token מופיע → כמה הוא חשוב

🚀 שלב 3: tokenization

👉 לכל מילה:

מחשבים את כל הפירוקים האפשריים
👉 ובחרים את זה עם ההסתברות הכי גבוהה

♦ דוגמה

מילה:

None

pug

אפשרויות:

None

["p", "u", "g"]

["p", "ug"]

["pu", "g"]

👉 בוחרים את הכי סביר

אינטואיציה 🧠

הכי פחות tokens → הכי טוב (בד"כ)

איך מוצאים את הפירוק הכי טוב? 🔥

משתמשים ב-Viterbi algorithm 👉

מה זה אומר? 💡

בונים את הפתרון הכי טוב צעד-צעד

דוגמה ♦

None

`["unhug" → ["un", "hug`

שלב 4: חישוב loss 🚀

לכל המילים: 👉

]
((loss = -\log(P(word
[

אינטואיציה 💡

כמה טוב ה-tokenizer מסביר את הדאטה

שלב 5: הסרת tokens 🔥

לכל token: 👉

- מסירים אותו
- בודקים כמה ה-loss גדל

החלטה 🧠

מסירים tokens ש: 👉

- כמעט לא משפיעים על loss

חוזרים על התהליך 🔄

כל פעם: 👉

- מסירים % מהtokens
- עד שמגיעים לגודל vocab הרצוי

חשוב מאוד ⚠️

לא מסירים אותיות בסיס 👉
(כדי שתמיד אפשר לפרק מילים)

תכונה חשובה 🔥
SentencePiece

לא מניח שיש רווחים 👉

דוגמה:

None

"Hello world"

הופך ל: 👉

None

Hello _world_

אינטואיציה 🧠

הרווח הוא חלק מהטוקן

יתרון גדול 🔥

עובד טוב לשפות בלי רווחים ✓
גמיש מאוד ✓

סיכום תהליך 📊

פעולה	שלב
vocab גדול	init
חישוב הסתברויות	prob
כל האפשרויות	tokenize
מדידת איכות	loss
הסרת tokens	remove
עד גודל מתאים	repeat

השוואה בין האלגוריתמים 📊

Unigram	WordPiece	BPE	תכונה
remove	build	build	כיוון
loss	score	freq	החלטה
הכי גבוה	גבוה	בינוני	גמישות
גבוהה	בינונית	נמוכה	מורכבות



takeaway הכי חשוב 🧠

Unigram בוחר את הפירוק הכי "סביר הסתברותית"

משפט זהב 🔥

"Unigram doesn't build tokens — it eliminates the unnecessary ones"

חיבור לכל הפרק 🚀

עכשיו יש לך:

- BPE → GPT
- WordPiece → BERT
- Unigram → T5

👉 שזה שלושת האלגוריתמים המרכזיים בעולם ה-NLP

פרק 6.9

👉 איך לבנות tokenizer מאפס לגמרי
(לא רק לאמן — אלא להרכיב אותו שלב-שלב)

התמונה הגדולה 🧠

כל tokenizer בנוי מ-4 שלבים:

None

Text → Normalization → Pre-tokenization → Model →
Post-processing

הרעיון המרכזי 🔥

tokenizer הוא pipeline
👉 וכל שלב אפשר להחליף / להתאים

אבני הבניין

הספרייה נותנת מודולים:

חלק	תפקיד
normalizers	ניקוי טקסט
pre_tokenizers	פירוק ראשוני
models	BPE / WordPiece / Unigram
trainers	אימון
post_processors	הוספת special tokens
decoders	חזרה לטקסט

אינטואיציה 

tokenizer = Lego
את בונה אותו מחלקים 

dataset :1 שלב 

Python

```
(...)load_dataset
```

מביאים corpus לאימון 

אינטואיציה 

tokenizer לומד מהדאטה שלך

שלב 2: יצירת Tokenizer 🚀

Python

```
((...))Tokenizer(models.WordPiece
```

בחרים אלגוריתם 🙌

שלב 3: Normalization 🔥

Python

```
()normalizers.Lowercase
```

או:

Python

```
([...])Sequence
```

מה עושים פה? 💡

- lowercase
- remove accents
- unicode normalization

שלב 4: Pre-tokenization 🚀

Python

```
()Whitespace
```

או:

Python

```
([...])Sequence
```

דוגמה 💡

None

"Let's test"

["Let", "'", "s", "test"] →

אינטואיציה 🧠

חיתוך גס לפני פירוק עמוק

שלב 5: Training 🚀

Python

```
(...)trainer = WordPieceTrainer  
(...)tokenizer.train_from_iterator
```

חשוב 💡

צריך להוסיף:

Python

```
[... , "[CLS]", "[SEP]"]
```

למה? ⚠️

אחרת הם לא יהיו ב-vocab 🙅

שלב 6: encode 🚀

Python

```
(...)tokenizer.encode
```

דוגמה:

None

```
"tokenizer"  
["tok", "##eni", "##zer"] →
```

Post-processing שלב 7: 🚀

מוסיפים special tokens 👉

דוגמה (BERT):

None

```
[CLS] sentence [SEP]
```

קוד:

Python

```
(...)TemplateProcessing
```

Decoder שלב 8: 🚀

Python

```
(...)decoders.WordPiece
```

תפקיד 💡

tokens → text מחזיר 🙌

שלב 9: שמירה 🔥

Python

```
("tokenizer.save("tokenizer.json
```

שלב 10: שימוש ב-Transformers 🚀

Python

```
(...)PreTrainedTokenizerFast
```

אינטואיציה 🧠

עכשיו זה tokenizer "אמיתי" שאפשר להשתמש בו במודל

שלושת הסוגים 🔥

(BERT (WordPiece 💎

- lowercase
- Whitespace
- [CLS], [SEP]

(GPT-2 (BPE 💎

- Byte-level
- UNK אין
- <|special: <|endoftext

(XLNet (Unigram ♦

- SentencePiece
- metaspase
- <cls> בסוף (!)

השוואה 📊		
מודל	אלגוריתם	מיוחד
BERT	WordPiece	##
GPT-2	BPE	byte-level
XLNet	Unigram	—

takeaway הכי חשוב 🧠

tokenizer זה מערכת מודולרית
כל חלק משפיע על התוצאה 👉

פרק 6.10

סיימת את כל הפרק על Tokenizers 🎉
ועכשיו מסכמים מה את אמורה לדעת

התמונה הגדולה 🧠

את לא רק משתמשת ב-tokenizer
את מבינה איך הוא עובד מבפנים 👉

מה למדת (פירוק אמיתי) 🔥

1) לאמן tokenizer חדש

EN: Train a new tokenizer using an old one
HE: לאמן tokenizer חדש על בסיס tokenizer קיים

💡 אינטואיציה:

לא מתחילים מאפס — לוקחים מבנה קיים ומעדכנים אותו לדאטה שלך

2) לעבוד עם offsets

EN: Map tokens back to original text using offsets
HE: למפות טוקנים חזרה לטקסט המקורי בעזרת offsets

💡 אינטואיציה:

המודל עובד על tokens —
אבל אנחנו צריכים תשובות על טקסט אמיתי

3) להבין את האלגוריתמים

EN: Know differences between BPE, WordPiece, Unigram
HE: להבין את ההבדלים בין BPE, WordPiece ו-Unigram

💡 סיכום מהיר:

אלגוריתם	רעיון
BPE	לפי תדירות
WordPiece	לפי score
Unigram	מסיר tokens

4) לבנות tokenizer לבד

EN: Build your own tokenizer from blocks
HE: לבנות tokenizer משלך מחלקים שונים

אינטואיציה: 💡

tokenizer = pipeline מודולרי

5 להשתמש בו עם Transformers

EN: Use tokenizer inside 😊 Transformers
HE: להשתמש ב-tokenizer בתוך Transformers

אינטואיציה: 💡

ה-tokenizer הוא חלק מהמודל

סיכום בטבלה 📊	
מה זה נותן לך	יכולת
התאמה לדאטה	training
חיבור לטקסט	offsets
הבנה עמוקה	algorithms
שליטה מלאה	building
שימוש אמיתי	integration

takeaway הכי חשוב 🧠

tokenizer זה לא preprocessing קטן
👉 זה חלק קריטי מהמודל

פרק 6.11

שאלה 1

EN: When should you train a new tokenizer?
HE: מתי צריך לאמן tokenizer חדש?

- A.
When your dataset is similar to that used by an existing pretrained model, and you want to pretrain a new model
כאשר ה-dataset שלך דומה לזה של מודל קיים, ואת רוצה לאמן מודל חדש מאפס
- B.
When your dataset is similar to that used by an existing pretrained model, and you want to fine-tune a new model using this pretrained model
כאשר ה-dataset שלך דומה לזה של מודל קיים, ואת רוצה לבצע fine-tuning למודל קיים
- C.
When your dataset is different from the one used by an existing pretrained model, and you want to pretrain a new model
כאשר ה-dataset שלך שונה מזה של מודל קיים, ואת רוצה לאמן מודל חדש מאפס
- D.
When your dataset is different from the one used by an existing pretrained model, but you want to fine-tune a new model using this pretrained model
כאשר ה-dataset שלך שונה מזה של מודל קיים, אבל את עדיין רוצה לבצע fine-tuning למודל קיים
-

שאלה 2

EN: What is the advantage of using a generator of lists of texts compared to a list of lists of texts when using `train_new_from_iterator()`?

HE: מה היתרון של שימוש ב-generator של רשימות טקסטים לעומת רשימה של רשימות טקסטים ב-`train_new_from_iterator()`?

- A.
That's the only type the method accepts
זה הסוג היחיד שהפונקציה מקבלת
- B.
You will avoid loading the whole dataset into memory at once
לא תצטרכי לטעון את כל ה-dataset לזיכרון בבת אחת
- C.
This will allow the 🧑🏻 Tokenizers library to use multiprocessing
זה מאפשר לספרייה להשתמש ב-multiprocessing

D.
The tokenizer you train will generate better texts
ה-tokenizer ייצר טקסטים טובים יותר

שאלה 3

EN: What are the advantages of using a "fast" tokenizer?
HE: מהם היתרונות של שימוש ב-fast tokenizer?

A.
It can process inputs faster than a slow tokenizer when you batch lots of inputs together
הוא יכול לעבד קלטים מהר יותר כאשר מעבדים הרבה דוגמאות יחד

B.
Fast tokenizers always tokenize faster than their slow counterparts
fast tokenizers תמיד מהירים יותר

C.
It can apply padding and truncation
הוא יכול לבצע padding ו-truncation

D.
It has additional features allowing mapping from tokens to spans in the original text
יש לו יכולות נוספות למיפוי בין tokens לבין המיקום שלהם בטקסט המקורי

שאלה 4

EN: How does the token-classification pipeline handle entities that span over several tokens?
HE: איך pipeline של token-classification מטפל בישויות שמתפרסות על כמה tokens?

A.
Entities with the same label are merged into one entity
ישויות עם אותו label מתמזגות לישות אחת

B.
There is a label for the beginning of an entity and a label for the continuation of an entity
יש label לתחילת ישות ו-label להמשך ישות

C.
If in a given word the first token has the entity label, the whole word is considered to have that entity
אם הטוקן הראשון של מילה מסומן, כל המילה נחשבת מסומנת

D.
When a token has the label of a given entity, any following token with the same label is

considered part of it, unless it starts a new entity
כאשר token מסומן בישות מסוימת, טוקנים עוקבים עם אותו label נחשבים לחלק ממנה, אלא אם התחילה
ישות חדשה

שאלה 5

EN: How does the question-answering pipeline handle long contexts?
HE: איך pipeline של question-answering מטפל בקטעי טקסט ארוכים?

- A.
It truncates the long context
הוא פשוט חותך את הטקסט
 - B.
It splits the context into several parts and averages the results
הוא מחלק לחלקים ומחשב ממוצע
 - C.
It splits the context into several parts with overlap and finds the maximum score for an answer in each part
הוא מחלק לחלקים עם overlap ומחפש את התשובה עם הציון הגבוה ביותר בכל חלק
 - D.
It splits the context into several parts without overlap
הוא מחלק לחלקים בלי overlap
-

שאלה 6

EN: What is normalization?
HE: מה זה normalization?

- A.
Any cleanup the tokenizer performs on the text in the initial stages
כל ניקוי ראשוני שה-tokenizer מבצע על הטקסט
- B.
A data augmentation technique that makes the text more normal by removing rare words
טכניקת augmentation שמסירה מילים נדירות
- C.
The final post-processing step where special tokens are added
שלב סופי שבו מוסיפים special tokens
- D.
Making embeddings have mean 0 and std 1
הפיכת embeddings לבעלי ממוצע 0 וסטיות תקן 1

שאלה 7

EN: What is pre-tokenization for a subword tokenizer?
HE: מה זה pre-tokenization ב-subword tokenizer?

- A.
The step before tokenization where data augmentation is applied
שלב augmentation לפני tokenization
- B.
The step before tokenization where cleanup operations are applied
שלב הניקוי לפני tokenization
- C.
The step before the tokenizer model is applied, used to split input into words
שלב לפני הפעלת מודל הטוקניזציה, שבו מפצלים את הקלט למילים
- D.
The step before the tokenizer model is applied, used to split input into tokens
שלב שבו כבר מפצלים לטוקנים

שאלה 8

EN: Select the sentences that apply to the BPE model of tokenization.
HE: בחרי את המשפטים הנכונים לגבי BPE.

- A.
BPE starts with a small vocabulary and learns merge rules
BPE מתחיל מ-vocabulary קטן ולומד merge rules
- B.
BPE starts with a big vocabulary and progressively removes tokens
BPE מתחיל מ-vocabulary גדול ומסיר tokens
- C.
BPE learns by merging the most frequent pair of tokens
BPE לומד על ידי מיזוג הזוג הכי נפוץ
- D.
BPE learns by maximizing a score that prefers frequent pairs with rare individual parts
BPE לומד לפי score שמעדיף זוגות תכופים עם חלקים נדירים
- E.
BPE tokenizes by splitting into characters and then applying merge rules
BPE מפצל לאותיות ואז מפעיל merge rules

F.
BPE tokenizes by finding the longest subword from the beginning
BPE בוחר את תת־המילה הארוכה ביותר מההתחלה

שאלה 9

EN: Select the sentences that apply to the WordPiece model of tokenization.
HE: בחר את המשפטים הנכונים לגבי WordPiece.

- A.
WordPiece starts with a small vocabulary and learns merge rules
WordPiece מתחיל מ־vocab קטן ולומד merge rules
- B.
WordPiece starts with a big vocabulary and progressively removes tokens
WordPiece מתחיל מ־vocab גדול ומסיר tokens
- C.
WordPiece learns by merging the most frequent pair
WordPiece בוחר את הזוג הכי נפוץ
- D.
WordPiece learns by maximizing a score that prefers frequent pairs with rare individual parts
WordPiece בוחר לפי score שמעדיף זוגות תכופים עם חלקים נדירים
- E.
WordPiece tokenizes by finding the most likely segmentation according to the model
WordPiece מחפש את הפירוק הכי הסתברותי
- F.
WordPiece tokenizes by finding the longest subword from the beginning, then repeating
WordPiece בוחר את תת־המילה הארוכה ביותר מההתחלה, ואז ממשיך
-

שאלה 10

EN: Select the sentences that apply to the Unigram model of tokenization.
HE: בחר את המשפטים הנכונים לגבי Unigram.

- A.
Unigram starts with a small vocabulary and learns merge rules
Unigram מתחיל מ־vocab קטן ולומד merge rules
- B.
Unigram starts with a big vocabulary and progressively removes tokens
Unigram מתחיל מ־vocab גדול ומסיר tokens

C.

Unigram adapts its vocabulary by minimizing a loss over the whole corpus
Unigram מתאים את ה-vocab על ידי מזעור loss על כל הקורפוס

D.

Unigram adapts its vocabulary by keeping the most frequent subwords
Unigram פשוט שומר את תתי-המילים הכי נפוצים

E.

Unigram tokenizes words by finding the most likely segmentation according to the model
Unigram מפצל לפי הפירוק הכי הסתברותי

F.

Unigram tokenizes words by splitting into characters and then applying merge rules
Unigram מפצל לאותיות ואז מפעיל merge rules

תשובות מפורטות + הסברים 

שאלה 1 — תשובה נכונה: C

למה?

Tokenizer חדש מאמנים כשיש פער משמעותי בין הדאטה שלך לבין הדאטה שעליו אומן tokenizer קיים, ואת רוצה לאמן מודל חדש מאפס.

למה לא האחרות?

- A: אם הדאטה דומה, לרוב לא צריך tokenizer חדש.
- B: ב-fine-tuning משתמשים בדרך כלל באותו tokenizer של המודל הקיים.
- D: גם אם הדאטה שונה, ב-fine-tuning בדרך כלל לא מחליפים tokenizer, כי המודל כבר תלוי בו.

takeaway

Tokenizer חדש = בעיקר pretraining חדש על domain חדש.

שאלה 2 — תשובה נכונה: B

למה?

Generator טוען את הדאטה בהדרגה, במקום לטעון את כולו לזיכרון בבת אחת.

למה זה חשוב?

ב-datasets גדולים, אם תשתמשי ברשימה מלאה:

- תעמיסי על ה-RAM
- עלול לקרוס הזיכרון

למה לא האחרות?

- A: הפונקציה לא מוגבלת רק ל-generator.
- C: generator לא זה מה שמאפשר multiprocessing.
- D: הוא לא משפר את איכות הטקסטים, רק את היעילות בזיכרון.

takeaway

Generator = יעילות בזיכרון.

שאלה 3 — תשובות נכונות: A, D

A נכון

Fast tokenizer באמת מהיר יותר כאשר עובדים על batch גדול.

D נכון

יש לו יכולות נוספות כמו:

- `offset_mapping`
- `word_ids()`
- מיפוי `token ↔ text span`

למה B לא נכון?

כי לא תמיד fast tokenizer מהיר יותר על קלט יחיד קטן. היתרון בולט בעיקר ב-batching.

למה C לא נכון?

Padding ו-truncation קיימים גם ב-slow tokenizer, זה לא היתרון הייחודי של fast tokenizer.

takeaway

= Fast tokenizer

מהירות ב-batch + features מתקדמים של מיפוי.

שאלה 4 — תשובות נכונות: B, D

B נכון

יש labels בסגנון:

- B-PER = תחילת ישות
- I-PER = המשך ישות

D נכון

אם tokens רצופים שייכים לאותה ישות, מאחדים אותם יחד, אלא אם יש סימון שמתחילה ישות חדשה.

למה A לא מדויק?

כי לא כל tokens עם אותו label מתמזגים אוטומטית בכל מצב. צריך גם לבדוק רציפות והאם התחילה ישות חדשה.

למה C לא נכון?

לא מספיק שהטוקן הראשון של מילה מסומן כדי "לצבוע" אוטומטית את כל המילה. המערכת עובדת על tokens ו-labels, לא על המילה כמקשה אחת.

takeaway

Token classification עובד עם B-/I- labels ורצפים של tokens.

שאלה 5 — תשובה נכונה: C

למה?

ב-QA pipeline:

- אם ה-context ארוך מדי
- מחלקים אותו ל-chunks
- עם overlap
- מריצים חיפוש תשובה בכל chunk
- בוחרים את התשובה עם הציון הטוב ביותר

למה overlap חשוב?

כדי לא "לחתוך" תשובה בדיוק בנקודת הפיצול.

למה לא האחרות?

- A: זה היה מאבד תשובות בסוף הטקסט.
- B: לא עושים average.
- D: בלי overlap עלולים לפספס תשובות.

takeaway

QA pipeline = chunking + overlap + best span selection

שאלה 6 — תשובה נכונה: A

למה?

Normalization הוא שלב הניקוי הראשוני של הטקסט:

- lowercase
- remove accents
- Unicode normalization
- הסרת רווחים מיותרים

למה לא האחרות?

- B: זה לא augmentation.
- C: זה post-processing, לא normalization.
- D: זה normalization סטטיסטי של embeddings, לא של טקסט.

takeaway

Normalization = ניקוי אחיד של הטקסט לפני הטוקניזציה.

שאלה 7 — תשובה נכונה: C

למה?

Pre-tokenization הוא השלב שבו מחלקים את הטקסט ליחידות ראשוניות, בדרך כלל מילים או חתיכות דמויות-מילה, לפני שמודל הטוקניזציה (BPE / WordPiece / Unigram) מופעל.

למה לא האחרות?

- A: לא augmentation.
- B: זה normalization, לא pre-tokenization.
- D: עוד לא מפצלים ל-subtokens הסופיים.

takeaway

Pre-tokenization = פיצול ראשוני למילים/יחידות גסות.

שאלה 8 — תשובות נכונות: A, C, E

A נכון

BPE מתחיל מ-vocab קטן.

C נכון

הוא בוחר בכל שלב את הזוג הכי נפוץ.

E נכון

tokenization: בזמן

- מפרקים לאותיות / יחידות בסיס
- ואז מפעילים את merge rules

למה האחרות לא?

- B: זה Unigram, לא BPE.
- D: זה WordPiece, לא BPE.
- F: "longest subword from the beginning" זה WordPiece.

takeaway

BPE =

small vocab + frequent merges + apply merges in order

שאלה 9 — תשובות נכונות: A, D, F

A נכון

WordPiece מתחיל גם הוא מ־vocab קטן.

D נכון

הוא בוחר merges לפי score, לא רק לפי frequency.

F נכון

בזמן tokenization הוא מחפש את תת־המילה הארוכה ביותר מההתחלה שקיימת ב־vocab.

למה האחרות לא?

- B: זה Unigram.
- C: זה BPE.
- E: "most likely segmentation" זה Unigram.

takeaway

WordPiece =

small vocab + score-based merges + longest-match encoding

שאלה 10 — תשובות נכונות: B, C, E

B נכון

Unigram מתחיל מ-vocab גדול ואז מסיר tokens.

C נכון

הוא מתאים את ה-vocab לפי loss על כל הקורפוס.

E נכון

tokenization הוא בוחר את הפירוק הכי הסתברותי.

למה האחרות לא?

- A: לא נכון, זה לא מתחיל קטן.
- D: הוא לא רק שומר frequent subwords, אלא בודק loss.
- F: זה יותר מזכיר BPE.

שלושת הדברים הכי חשובים לזכור מהפרק 🧠

1. Tokenizer חדש מאמנים כשעושים pretraining חדש על data/domain שונה.
2. Fast tokenizer נותן offsets ו-mapping, לא רק מהירות.
3. BPE / WordPiece / Unigram נבדלים בעיקר בדרך שבה הם בוחרים subwords.

פרק 7

פרק 7.1

בפרק 3:
👉 למדת איך לעשות fine-tuning ל-classification

אבל עכשיו:

🔥 עוברים למשימות NLP אמיתיות

🧠 מה הולכים ללמוד בפרק הזה?

המשימות המרכזיות:

Token classification¹

👉 זיהוי ישויות (NER)

Masked Language Modeling²

👉 כמו BERT
👉 לנחש מילה חסרה

Summarization³

👉 סיכום טקסט

Translation⁴

👉 תרגום בין שפות

Causal Language Modeling⁵

👉 כמו GPT-2
👉 חיזוי המילה הבאה

Question Answering⁶

👉 מציאת תשובה בתוך טקסט

אינטואיציה הכי חשובה 🧠

כל ה-LLMs בנויים מהמשימות האלה

הרעיון המרכזי 🔥

הפרק הזה =
המקום שבו הכל מתחבר יחד 🙌

מה משתמשים בו?

- Trainer (מפרק 3) 🧐 •
 - Datasets (פרק 5) 🧐 •
 - Tokenizers (פרק 6) 🧐 •
 - Hub (פרק 4) 🧐 •
-

אינטואיציה 🧠

עד עכשיו למדת כל חלק בנפרד
עכשיו מחברים הכל למערכת אחת 🙌

שתי דרכים לאמן מודל 🚀

Trainer API ♦

קל ✓
מהיר ✓
פחות שליטה ✓

Accelerate עם Training loop ♦

שליטה מלאה ✓
יותר מורכב ✓

איך לבחור? 🧠

מה לבחור	מצב
Trainer	רוצה מהר
Accelerate	רוצה שליטה

חשוב מאוד 🔥

כל חלק בפרק:

עצמאי 👉

אפשר:

- ללמוד לפי הסדר
- או לקפוץ לנושא ספציפי

למה יש חזרות בקוד? 🧠

בכוונה! 👉

כדי:

- שתוכלי ללמוד כל נושא לבד
- בלי תלות בשאר הפרק

פרק 7.2

Token classification היא משימה שבה נותנים תווית לכל טוקן/מילה במשפט.

דוגמאות:

- NER: לזהות ישויות כמו אדם, מקום, ארגון
- POS tagging: לזהות סוג מילה, כמו noun / verb / adjective
- Chunking: לזהות אילו מילים שייכות לאותו chunk

כלומר, במקום:

- משפט אחד → תווית אחת

יש לנו:

- משפט אחד → תווית לכל מילה

(2) ההבנה הכי בסיסית

אם יש משפט:

. EU rejects German call to boycott British lamb

אנחנו רוצים שלכל מילה תהיה תווית. למשל:

- EU = ארגון
- German = משהו מסוג MISC
- מילים אחרות = O, כלומר לא ישות

כלומר המודל לא שואל "מה המשמעות של המשפט כולו?"
הוא שואל "מה התווית של כל מילה?"

(3) סוגי התוויות

בדאטהסט הזה יש תוויות כאלה:

- O = לא ישות
- B-PER = תחילת ישות מסוג אדם
- I-PER = המשך ישות מסוג אדם
- B-ORG = תחילת ארגון
- I-ORG = המשך ארגון
- B-LOC = תחילת מקום
- I-LOC = המשך מקום
- B-MISC = תחילת ישות אחרת
- I-MISC = המשך ישות אחרת

למה צריך גם B וגם I?

כי יש ישויות שמכילות כמה מילים.

למשל:

- European Union
 - European = B-ORG
 - Union = I-ORG

או:

- Werner Zwingmann
 - Werner = B-PER
 - Zwingmann = I-PER

כלומר:

- B = Begin
- I = Inside

(4) הדאטהסט שבו משתמשים

הם משתמשים בדאטהסט CoNLL-2003.

יש בו 3 splits:

- train
- validation
- test

וכל דוגמה מכילה:

- tokens = רשימת מילים
- ner_tags = רשימת תוויות ל-NER
- pos_tags = רשימת תוויות ל-POS
- chunk_tags = רשימת תוויות ל-chunking

חשוב להבין:

הקלט פה כבר מחולק למילים.

כלומר הוא לא מגיע כ-string ארוך, אלא כבר בתור רשימת מילים.

(5) הבעיה המרכזית בכל Token Classification

ופה מגיע הקטע שהכי חשוב להבין:

ה-labels ניתנים למילים
אבל המודל של BERT עובד על tokens / subwords

למשל:

- המילה `lamb`
- עשויה להתפצל ל:
 - `la`
 - `mb##`

אז פתאום יש בעיה:

- במקור יש לי label אחד עבור `lamb`
- אחרי tokenization יש לי שני tokens

כלומר:

יש mismatch בין מספר המילים עם labels לבין מספר ה-tokens שהמודל באמת רואה.

וזו הבעיה הכי חשובה בפרק הזה.

(6) איך פותרים את הבעיה?

עושים alignment בין labels לבין tokens.

הרעיון:

- כל token צריך לקבל label
- special tokens כמו `[CLS]` ו-`[SEP]` לא מייצגים מילה אמיתית, אז ניתן להם `-100`
- אם מילה התפצלה לכמה subtokens:
 - הראשון מקבל את התווית המקורית
 - ההמשכים מקבלים אותה תווית, אבל אם זה `-B` אז משנים ל-`-I`...

למה `-100`?

כי בפונקציית loss של PyTorch, ברירת המחדל היא להתעלם מ-`-100`.

כלומר:

- על special tokens לא מחשבים loss
- על padding labels לא מחשבים loss

(7) מה זה `word_ids()` ולמה זה חשוב?

אחרי שעושים tokenization עם tokenizer מהיר (fast tokenizer), אפשר לקבל מיפוי:

- כל token → לאיזו מילה מקורית הוא שייך

למשל:

[None, 0, 1, 2, 3, 4, 5, 6, 7, 7, 8, None]

פירוש:

- None = special token
- token אחד שייך למילה 0
- token אחר שייך למילה 1
- שני tokens שייכים למילה 7 (lamb שהתפצלה)

זה מה שמאפשר לנו ליישר labels לטוקנים.

(8) למה צריך Data Collator מיוחד?

במשימות רגילות אפשר להשתמש ב-DataCollatorWithPadding, אבל פה זה לא מספיק.

למה?

כי פה לא רק ה-input צריך padding — גם ה-labels צריכים padding.

אם משפט אחד ארוך יותר מאחר, צריך:

- לרפד את ה-input ids
- לרפד את attention mask
- וגם לרפד את ה-labels

ועושים את זה עם:

DataCollatorForTokenClassification

וה-padding של labels יהיה עם -100.

(9) למה metric רגיל לא מספיק?

בפרק הזה לא משתמשים רק ב-accuracy רגיל.

משתמשים ב-seqeval.

למה?

כי במשימות כמו NER חשוב למדוד נכון ישויות שלמות:

- Precision
- Recall
- F1

הוא מסתכל על הרצף של התוויות ולא רק על התאמה טכנית של כל token בנפרד.

(10) למה צריך `id2label` ו-`label2id`?

כשמגדירים מודל ל-token classification, חשוב שהמודל ידע:

- איזה מספר מייצג איזה label
- ואיך להפוך label לשם ולהפך

זה חשוב גם לשמירה של המודל וגם לשימוש מאוחר יותר ב-pipeline וב-Hub.

(11) למה צריך לבדוק `model.config.num_labels`?

כי אם מספר ה-labels לא מתאים לדאטהסט, עלולה לצאת שגיאה קשה ומבלבלת כמו:
`CUDA error: device-side assert triggered`

זו אחת הטעויות הכי נפוצות.

Python

```
from datasets import load_dataset

) from transformers import
    ,AutoTokenizer
    ,DataCollatorForTokenClassification
    ,AutoModelForTokenClassification
    ,TrainingArguments
    ,Trainer
    ,pipeline

(
    import evaluate
    import numpy as np
```

```

#
=====

Load dataset .1 #

#
=====

("raw_datasets = load_dataset("conll2003

#
=====

Inspect label names .2 #

#
=====

["ner_feature = raw_datasets["train"].features["ner_tags
label_names = ner_feature.feature.names

(print("Label names
(print(label_names

#
=====

Inspect one sample .3 #

```

```

=====
#
=====

["example_tokens = raw_datasets["train"][0]["tokens
["example_labels = raw_datasets["train"][0]["ner_tags

        (:print("\nExample tokens
            (print(example_tokens

        (:print("\nExample label ids
            (print(example_labels

        (:print("\nExample labels as names
        ([print([label_names[i] for i in example_labels

=====
#
=====

Create tokenizer .4 #

#
=====

"model_checkpoint = "bert-base-cased

tokenizer =
(AutoTokenizer.from_pretrained(model_checkpoint

```

```

        ("?print("\nIs tokenizer fast
            (print(tokenizer.is_fast

#
=====
=

        Tokenize one example .5 #

#
=====
=

        inputs = tokenizer(example_tokens,
            (is_split_into_words=True

        (":print("\nTokenized tokens
            (())print(inputs.tokens

        (":print("\nWord ids
            (())print(inputs.word_ids

#
=====
=

        Align labels with tokens .6 #

#
=====
=

:(def align_labels_with_tokens(labels, word_ids

```

```

        [] = new_labels
        current_word = None

        :for word_id in word_ids
        :if word_id != current_word
            current_word = word_id

            Special token #
            :if word_id is None
                (new_labels.append(-100
            :else
                ([new_labels.append(labels[word_id

            :elif word_id is None
                Special token #
                (new_labels.append(-100

            :else
                Same word as previous token #
                [label = labels[word_id

        Convert B-XXX to I-XXX for subsequent #
                                                    subtokens

        :if label % 2 == 1

```

```

label += 1

(new_labels.append(label

return new_labels

aligned_labels =
align_labels_with_tokens(example_labels,
                          (inputs.word_ids

):print("\nAligned labels
(print(aligned_labels

print("\nAligned labels as readable names (ignoring
      (":-100

print([label_names[i] if i != -100 else -100 for i in
      ([aligned_labels

#
=====
=

Tokenize full dataset and align labels .7 #

#
=====
=

:(def tokenize_and_align_labels(examples

```



```

        )tokenized_inputs = tokenizer
            ,["examples["tokens
                ,truncation=True
            is_split_into_words=True
                (

["all_labels = examples["ner_tags
                    [] = new_labels

:(for i, labels in enumerate(all_labels
(word_ids = tokenized_inputs.word_ids(i

new_labels.append(align_labels_with_tokens(labels,
((word_ids

tokenized_inputs["labels"] = new_labels
    return tokenized_inputs

)tokenized_datasets = raw_datasets.map
    ,tokenize_and_align_labels
        ,batched=True
    ,remove_columns=raw_datasets["train"].column_names
        (

```

```

=====
#
Data collator .8 #
=====
#
data_collator =
(DataCollatorForTokenClassification(tokenizer=tokenizer

batch = data_collator([tokenized_datasets["train"][i]
                        ([for i in range(2

(":print("\nBatch labels after padding
      (["print(batch["labels

=====
#
Metric .9 #
=====
#
("metric = evaluate.load("sequeval

:(def compute_metrics(eval_preds
    logits, labels = eval_preds

```

```

        (predictions = np.argmax(logits, axis=-1

                                ] = true_labels

[label_names[l] for l in label if l != -100]

                                for label in labels

                                [

                                ] = true_predictions

label_names[p] for (p, l) in zip(prediction, [
                                label) if l != -100

                                for prediction, label in zip(predictions,
                                                                (labels

                                                                [

                                                                )all_metrics = metric.compute
                                                                ,predictions=true_predictions
                                                                references=true_labels

                                                                (

                                                                } return

,["precision": all_metrics["overall_precision"

                                ,["recall": all_metrics["overall_recall"

                                ,["f1": all_metrics["overall_f1"

                                ,["accuracy": all_metrics["overall_accuracy"

```

```

{

#
=====
=

Build label mappings .10 #

#
=====
=

id2label = {i: label for i, label in
              {(enumerate(label_names
{()label2id = {label: i for i, label in id2label.items

#
=====
=

Load model .11 #

#
=====
=

)model = AutoModelForTokenClassification.from_pretrained
        ,model_checkpoint
        ,id2label=id2label
        ,label2id=label2id

(

(:print("\nNumber of labels in model

```

```

(print(model.config.num_labels

#
=====
=

Training arguments .12 #

#
=====
=

)args = TrainingArguments
,"output_dir="bert-finetuned-ner
,"evaluation_strategy="epoch
,"save_strategy="epoch
,"learning_rate=2e-5
,"num_train_epochs=3
,"weight_decay=0.01

(

#
=====
=

Trainer .13 #

#
=====
=

)trainer = Trainer

```

```

        ,model=model

        ,args=args

        ,["train_dataset=tokenized_datasets["train
        ,["eval_dataset=tokenized_datasets["validation

        ,data_collator=data_collator

        ,compute_metrics=compute_metrics

        ,tokenizer=tokenizer

        (

        #
=====
=

Train .14 #

#
=====
=

()trainer.train

#
=====
=

Inference with pipeline .15 #

#
=====
=

)token_classifier = pipeline

```

```

        , "token-classification"
        , model=model
        , tokenizer=tokenizer
        "aggregation_strategy="simple
    )

result = token_classifier("My name is Sylvain and I work
                           (.at Hugging Face in Brooklyn
                           (:print("\nPipeline result
                           (print(result

```

13) הסבר של הקוד, שלב אחרי שלב, בלי לפספס
טעינת dataset

Python

```
("raw_datasets = load_dataset("conll2003
```

טוען את הדאטהסט המוכן.

חילוץ שמות labels

Python

```
["ner_feature = raw_datasets["train"].features["ner_tags
label_names = ner_feature.feature.names
```

ה-labels בדאטה מאוחסנים כמספרים.
פה הופכים אותם לשמות כמו B-ORG, I-LOC וכן הלאה.

יצירת tokenizer

Python

```
tokenizer =  
(AutoTokenizer.from_pretrained(model_checkpoint
```

טוען tokenizer שמתאים ל-`bert-base-cased`.

tokenization של רשימת מילים

Python

```
inputs = tokenizer(example_tokens,  
                    (is_split_into_words=True
```

זה קריטי:

`is_split_into_words=True`
כי הקלט הוא כבר רשימת מילים, לא `string`.

קבלת `word_ids`

Python

```
()inputs.word_ids
```

נותן לנו מיפוי `word index` → `token`
וזו הבסיס ליישור ה-`labels`.

פונקציית `align_labels_with_tokens`

זו הפונקציה הכי חשובה:

- אם זה `token` חדש של מילה חדשה → תן את ה-`label` של אותה מילה
- אם זה `special token` → תן `-100`
- אם זה עוד `subtoken` של אותה מילה:
 - קח את ה-`label`
 - אם הוא `-B...`, שנה אותו ל-`-I...`

preprocessing מלא לכל הדאטהסט

Python

```
(...)tokenized_datasets = raw_datasets.map
```

מריץ את תהליך ה-tokenization + alignment על כל הדוגמאות.

data collator

Python

```
data_collator =  
(DataCollatorForTokenClassification(tokenizer=tokenizer
```

דואג לריפוד batch-ים בצורה נכונה גם ל-inputs וגם ל-labels.

metric

Python

```
("metric = evaluate.load("seqeval
```

זה metric סטנדרטי ל-NER / token classification.

compute_metrics

הפונקציה:

1. לוקחת logits
2. עושה `argmax`
3. מסירה את כל המקומות שבהם label הוא -100
4. ממירה ids לשמות labels
5. מחשבת precision / recall / f1 / accuracy

`label2id` ו-`id2label`

נותנים למודל את ההתאמה בין מספרים לשמות labels.

טעינת המודל

Python

```
model =  
(...)AutoModelForTokenClassification.from_pretrained
```

משתמשים במודל שמתאים למשימת token classification.

אימון עם Trainer

ה-Trainer מקבל:

- מודל
- training args
- train dataset
- validation dataset
- data collator
- metric function
- tokenizer

ואז:

Python

```
()trainer.train
```

pipeline בסוף

Python

```
(... , "pipeline("token-classification
```

מאפשר להשתמש במודל המאומן על טקסט חדש.

(14) הקטע האחרון בחומר — Accelerate עם custom training loop

החומר גם מראה דרך שנייה לאמן את אותו מודל, בלי Trainer, אלא עם Accelerate.

הרעיון שם:

- בונים DataLoader ידנית
- מגדירים optimizer ידנית

- משתמשים ב-`Accelerator`
- כותבים training loop ידנית
- עושים evaluation ידנית
- שומרים מודל ידנית

זו אותה משימה, פשוט יותר שליטה ויותר קוד.

השלבים שם:

1. בונים `train_dataloader` ו-`eval_dataloader`
2. מטענים מחדש את המודל
3. מגדירים `AdamW`
4. מעבירים הכל דרך `accelerator.prepare(...)`
5. מחשבים `num_training_steps`
6. מגדירים `lr_scheduler`
7. כותבים לולאת `training`
8. כותבים `evaluation loop`
9. משתמשים ב-`accelerator.pad_across_processes(...)`
10. אוספים `predictions` מכל התהליכים עם `accelerator.gather(...)`
11. שומרים ומעלים ל-Hub

למה צריך `pad_across_processes`?

כי ב-distributed training יכול להיות שלכל process יש `tensors` באורכים שונים, וצריך להשוות אותם לפני `.gather`.

למה צריך `unwrap_model`?

כי אחרי `accelerator.prepare()` המודל עטוף, וצריך להחזיר אותו לצורה המקורית לפני `.save_pretrained`.

פרק 7.3

♦ (1)

= Masked Language Modeling
מודל שמקבל משפט עם חור (`[MASK]`) וצריך לנחש את המילה החסרה

דוגמה:

None

[This is a great [MASK]

המודל יגיד:

- movie
- idea
- story

2 ♦

זה בעצם כמו:

👉 השלמת משפטים חכמה

אבל לא סתם:

- המודל לומד סטטיסטיקה של שפה
- מה סביר לבוא אחרי מה

💡 למה עושים fine-tuning?

יש מודל pretrained (כמו BERT)

אבל הוא למד על:

- Wikipedia
- ספרים

👉 אם הדאטה שלך שונה (למשל ביקורות סרטים)

אז:

👉 כדאי להתאים אותו לדומיין שלך

זה נקרא:

Domain Adaptation 🔥

3 ♦ בחירת מודל

משתמשים ב:

Python

AutoModelForMaskedLM

בדוגמה:

Python

```
"model_checkpoint = "distilbert-base-uncased"
```

למה DistilBERT? 🧠

- קטן יותר
- מהיר יותר
- כמעט אותה ביצועים

4 קוד — טעינת מודל וטוקנייזר

Python

```
from transformers import AutoModelForMaskedLM,
                          AutoTokenizer

"model_checkpoint = "distilbert-base-uncased"

model =
(AutoModelForMaskedLM.from_pretrained(model_checkpoint

tokenizer =
(AutoTokenizer.from_pretrained(model_checkpoint
```

הסבר 💡

- המודל יודע לנבא [MASK]
- tokenizer ממיר טקסט ל-tokens

Python

```
import torch

text = "This is a great [MASK]"

inputs = tokenizer(text, return_tensors="pt")
logits = model(**inputs).logits

mask_index = torch.where(inputs["input_ids"] ==
                           [tokenizer.mask_token_id])[1]

mask_logits = logits[0, mask_index]

top_tokens = torch.topk(mask_logits, 5,
                        dim=1).indices[0].tolist

for token in top_tokens:
    print(text.replace(tokenizer.mask_token,
                       tokenizer.decode([token])))
```

מה קורה פה? 🧠

1. עושים tokenize
2. מוצאים איפה [MASK]
3. לוקחים logits של המקום הזה
4. בוחרים top 5 מילים

Python

```
from datasets import load_dataset

imdb_dataset = load_dataset("imdb
```

הסבר 🧠

יש:

- (train (25K
- (test (25K
- (unsupervised (50K

אנחנו לא צריכים labels פה! 👉

preprocessing — tokenize (7 ♦

Python

```
:(def tokenize_function(examples
(["result = tokenizer(examples["text

:if tokenizer.is_fast
] = ["result["word_ids
result.word_ids(i) for i in
(["range(len(result["input_ids
[

return result
```

```

)tokenized_datasets = imdb_dataset.map
    ,tokenize_function
    ,batched=True
    ["remove_columns=["text", "label
(

```

?word_ids למה 🧠

בשביל whole word masking (בהמשך) 👉

8) בעיה חדשה ♦

הטקסטים:

- ארוכים
- באורכים שונים

פתרון 💡

מחברים הכל → ואז חותכים ל-chunks 👉

9) chunking ♦

Python

```
chunk_size = 128
```

```

:(def group_texts(examples
concatenated = {k: sum(examples[k], []) for k in
                  {()}examples.keys

```



```

(["total_length = len(concatenated["input_ids

total_length = (total_length // chunk_size) *
                                chunk_size

                                } = result

k: [t[i:i + chunk_size] for i in range(0,
                                [(total_length, chunk_size

                                ()for k, t in concatenated.items

                                {

                                ()result["labels"] = result["input_ids"].copy

                                return result

lm_datasets = tokenized_datasets.map(group_texts,
                                (batched=True

```

הסבר עמוק 🧠

למה מחברים טקסטים?

כדי:

👉 לא לאבד מידע בגלל truncation

למה labels = input_ids?

כי:

👉 המודל צריך לנחש מילים מתוך הקלט

Python

```

from transformers import DataCollatorForLanguageModeling

data_collator = DataCollatorForLanguageModeling
                    ,tokenizer=tokenizer
                    mlm_probability=0.15
                    (

```

מה הוא עושה? 🧠

מוסיף [MASK] רנדומלי 🙌

15% מהמילים:

- חלק מוחלפות ב-[MASK]
- חלק מוחלפות במילים אחרות
- חלק נשארות כמו שהן

זה הקסם של MLM 🔥

לא צריך להכין labels ידנית
זה נעשה תוך כדי training 🙌

Python

```

from transformers import TrainingArguments, Trainer

training_args = TrainingArguments

```

```

        , "output_dir="model
        , "evaluation_strategy="epoch
        , learning_rate=2e-5
        , weight_decay=0.01
    , per_device_train_batch_size=64
    , per_device_eval_batch_size=64

    (

        )trainer = Trainer
        , model=model
        , args=training_args
    , [ "train_dataset=lm_datasets["train
    , [ "eval_dataset=lm_datasets["test
        , data_collator=data_collator
        , tokenizer=tokenizer

    (

        )trainer.train

```

הסבר 🧠

Trainer עושה:

- forward
- loss
- backward
- update

Perplexity — מדידה (12 ♦

Python

```
import math

()eval_results = trainer.evaluate

([ "print(math.exp(eval_results["eval_loss
```

אינטואיציה 🧠

Perplexity = כמה המודל "מבולבל"

- נמוך = טוב
- גבוה = רע

Whole Word Masking (מתקדם) (13 ♦

במקום:

tokens לפי masking 👉

עושים:

masking לפי מילים 👉

קוד:

Python

```
:(def whole_word_masking_data_collator(features

import numpy as np

import collections
```

```

        :for feature in features

        ("word_ids = feature.pop("word_ids

(mapping = collections.defaultdict(list

        current_word = None

        word_index = -1

:(for idx, word_id in enumerate(word_ids

        :if word_id is not None

        :if word_id != current_word

        current_word = word_id

        word_index += 1

(mapping[word_index].append(idx

((mask = np.random.binomial(1, 0.2, len(mapping

        ["input_ids = feature["input_ids

        ["labels = feature["labels

        (new_labels = [-100] * len(labels

        :[for word_id in np.where(mask)[0

        :[for idx in mapping[word_id

        [new_labels[idx] = labels[idx

```

```
input_ids[idx] = tokenizer.mask_token_id
```

```
feature["labels"] = new_labels
```

```
return features
```

למה זה טוב? 🧠

המודל לומד מילים שלמות
ולא חצאי tokens

14 ♦ שימוש במודל

Python

```
from transformers import pipeline
```

```
)mask_filler = pipeline
```

```
, "fill-mask"
```

```
"model="your-model"
```

```
(
```

```
("[mask_filler("This is a great [MASK
```

נושא	רעיון
MLM	לנחש מילים חסרות
[MASK]	token חסר
labels	copy של input_ids
data collator	מוסיף masking
chunking	חותך טקסטים
perplexity	מדד ביצועים
domain adaptation	התאמה לדאטה

פרק 7.4



= Translation
לקחת משפט בשפה אחת → ולהפיק משפט בשפה אחרת
לדוגמה:

None

English: Default to expanded threads

French: Par défaut, développer les fils de discussion



זו משימת:
Sequence → Sequence 🙌

כלומר:

- קלט = רצף מילים
- פלט = רצף מילים אחר

זה דומה ל: 💡

- סיכום (Summarization)
- QA גנרטיבי
- שינוי סגנון

ההבדל הגדול ממודלים רגילים 🔥

במקום:

None

`input` → `label` אחד

יש:

None

`input` → משפט חדש שלם

3) למה עושים fine-tuning? ♦

יש מודל מוכן (pretrained)
אבל:

לפעמים הוא לא מתאים לדאטה שלך 🙅

לדוגמה:

- המודל ישאיר "plugin" באנגלית
- אבל בדאטה שלך זה מתורגם

אז עושים model → fine-tuning מתאים לדומיין 🙅

KDE4 — דאטה 4 ♦

Python

```
from datasets import load_dataset

raw_datasets = load_dataset("kde4", lang1="en",
                             ("lang2="fr
```

מה יש בדאטה? 🧠

כל דוגמה:

Python

```
}

} : "translation"

, "... " : "en"

"... " : "fr"

{

{
```

חשוב 💡

זה לא:

Python

text + label

זה:

Python

input sentence + target sentence

5 split לדאטה ♦

Python

```
)split_datasets = raw_datasets["train"].train_test_split
```

```
train_size=0.9, seed=20
```

```
(
```

```
split_datasets["validation"] =  
    ("split_datasets.pop("test
```

למה? 🧠

כי אין validation → יוצרים לבד

tokenizer (6 ♦

Python

```
from transformers import AutoTokenizer

model_checkpoint = "Helsinki-NLP/opus-mt-en-fr"

tokenizer =
(AutoTokenizer.from_pretrained(model_checkpoint
```

preprocessing (7) — הכי חשוב פה ♦

Python

```
max_length = 128

:(def preprocess_function(examples

    inputs = [ex["en"] for ex in
               ["examples["translation

    targets = [ex["fr"] for ex in
               ["examples["translation

    )model_inputs = tokenizer
```

```

        ,inputs

        ,text_target=targets

        ,max_length=max_length

        truncation=True

    (

    return model_inputs

```

הסבר עמוק 🧠

inputs

משפטים באנגלית

targets

משפטים בצרפתית

הדבר הקריטי 🔥

Python

```
text_target=targets
```

👉 זה אומר:

- tokenizer אחד ל-input
- tokenizer אחר ל-labels

! אם לא עושים את זה

המודל לא ילמד טוב בכלל

apply preprocessing (8 ♦

Python

```
tokenized_datasets = split_datasets.map  
    , preprocess_function  
    , batched=True  
    , remove_columns=split_datasets["train"].column_names  
(
```

9) מודל ♦

Python

```
from transformers import AutoModelForSeq2SeqLM  
  
model =  
(AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint
```

למה Seq2Seq? 🧠

כ"י: input → output sequence 👉

Data Collator (10 📌)

Python

```
from transformers import DataCollatorForSeq2Seq

data_collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=model,
    )
```

מה הוא עושה? 🧠

- input-ל padding
- labels-ל padding
- labels padded עם -100
- יוצר decoder_input_ids

חשוב למבחן 🔥

labels padding = -100
👉 כדי שלא יכנס ל-loss

Metric — BLEU (11 📌)

Python

```
import evaluate
```

```
("metric = evaluate.load("sacrebleu
```

אינטואיציה 🧠

מודד:

👉 כמה התרגום דומה ל-target

טווח 📊

- 0 → גרוע
- 100 → מושלם

compute_metrics (12 💎

Python

```
import numpy as np
```

```
:(def compute_metrics(eval_preds
```

```
preds, labels = eval_preds
```

```

        :(if isinstance(preds, tuple)
            [preds = preds[0]

        )decoded_preds = tokenizer.batch_decode

            ,preds

            skip_special_tokens=True

            (

labels = np.where(labels != -100, labels,
                  (tokenizer.pad_token_id

        )decoded_labels = tokenizer.batch_decode

            ,labels

            skip_special_tokens=True

            (

```



```

        decoded_preds = [pred.strip() for pred in
                           decoded_preds

decoded_labels = [[label.strip()] for label in
                   decoded_labels

        )result = metric.compute

        ,predictions=decoded_preds

        references=decoded_labels

        (

{"return {"bleu": result["score

```

הסבר 🧠

1. decode tokens → text
2. מסיר -100
3. מחשב BLEU

Training (13 ♦

Python

```
from transformers import Seq2SeqTrainingArguments
```

```
)args = Seq2SeqTrainingArguments  
  
    , "model"  
  
    , learning_rate=2e-5  
  
    , per_device_train_batch_size=32  
  
    , per_device_eval_batch_size=64  
  
    , num_train_epochs=3  
  
    , weight_decay=0.01  
  
    , predict_with_generate=True  
  
    (
```

חשוב 🔥

Python

```
predict_with_generate=True
```

משתמש ב-generate() 👉
כמו inference אמיתי 👉

Trainer 💡

Python

```
from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer

, model

, args

, ["train_dataset=tokenized_datasets["train

, ["eval_dataset=tokenized_datasets["validation

, data_collator=data_collator

, tokenizer=tokenizer

, compute_metrics=compute_metrics

(
```

אימון ♦

Python

```
(trainer.train
```

הערכה ♦

Python

```
(trainer.evaluate(max_length=128
```

שימוש ♦

Python

```
from transformers import pipeline

translator = pipeline(
    "translation",
    model="your-model"

("translator("Default to expanded threads
```

מה קורה בפועל במודל? 🔥

בזמן training:

- משתמש ב-labels
- decoder רואה תשובות אמיתיות

בזמן inference:

- משתמש ב:

Python

generate()

- מנבא מילה אחרי מילה

נושא	רעיון
task	sequence → sequence
input	שפה 1
output	שפה 2
tokenizer	צריך גם input וגם target
text_target	חובה!
model	Seq2Seq
collator	padding + decoder
metric	BLEU
generate	inference

פרק 7.5

= Summarization

לקחת טקסט ארוך → ולהפוך אותו לתקציר קצר שמכיל את העיקר

דוגמה:

None

Review: מוצר לא טוב, קטן, יקר ולא שימושי...

Summary: Not impressed



זו אחת המשימות הכי קשות ב-NLP כי צריך:

- להבין טקסט ארוך
- להבין מה חשוב ומה לא
- לייצר טקסט חדש

זה בעצם: 🔥

👉 "לתרגם" טקסט → לגרסה קצרה שלו

ולכן זה גם:

👉 Sequence → Sequence

♦ סוגי summarization (3)

1. Extractive

לוקח משפטים מהטקסט

2. Abstractive (מה שאנחנו עושים)

מייצר טקסט חדש 👉

4 דאטה ♦

משתמשים ב-Amazon Reviews:

Python

```
from datasets import load_dataset

english_dataset = load_dataset("amazon_reviews_multi",
                                ("en

spanish_dataset = load_dataset("amazon_reviews_multi",
                                ("es
```

מה יש בדאטה? 🧠

כל דוגמה:

Python

```
    }

    , "...": "review_body"

    "...": "review_title"

    {
```

טריק חשוב 💡

משתמשים ב-title כ-summary! 👉

5 סינון דאטה (Books) ♦

Python

```
:(def filter_books(example
    ) return
    "example["product_category"] == "book
    or example["product_category"] ==
        "digital_ebook_purchase
    (
```

Python

```
(english_books = english_dataset.filter(filter_books
(spanish_books = spanish_dataset.filter(filter_books
```

6 חיבור דאטה (דו-לשוני) ♦

Python

```
from datasets import concatenate_datasets, DatasetDict
```



```

books_dataset = DatasetDict

for split in english_books.keys():
    books_dataset[split] = concatenate_datasets
        ([english_books[split], spanish_books[split]]

        books_dataset[split] =
            (books_dataset[split].shuffle(seed=42

```

למה? 🧠

כדי לאמן מודל רב-לשוני 👉

7) ניקוי דאטה חשוב 💡

Python

```

books_dataset = books_dataset.filter

lambda x: len(x["review_title"].split()) > 2

(

```

למה? 💡

titles קצרים מדי → מודל ילמד להחזיר מילה אחת

8 מודל — mT5 ♦

Python

```
from transformers import AutoTokenizer

model_checkpoint = "google/mt5-small"

tokenizer =
(AutoTokenizer.from_pretrained(model_checkpoint
```

למה mT5? 🧠

- רב-לשוני
- עובד text-to-text
- מתאים ל summarization

9 preprocessing ♦

Python

```
max_input_length = 512

max_target_length = 30
```

```

:(def preprocess_function(examples

    )model_inputs = tokenizer

    ,["examples["review_body

    ,max_length=max_input_length

    truncation=True

    (

    )labels = tokenizer

    ,["examples["review_title

    ,max_length=max_target_length

    truncation=True

    (

    ["model_inputs["labels"] = labels["input_ids

    return model_inputs

```

input

review_body → טקסט ארוך

labels

review_title → summary

חשוב 🔥

- truncation גם ל-input וגם ל-labels
 - labels = tokenized summary
-

tokenize (10 🔹 הכל

Python

```
)tokenized_datasets = books_dataset.map  
  
    ,preprocess_function  
  
    batched=True  
  
    (
```

Metric — ROUGE (11 🔹

Python

```
import evaluate  
  
("rouge_score = evaluate.load("rouge
```

אינטואיציה 🧠

מודד:

👉 overlap בין summary אמיתי לנוצר

מרכיבים 📊

- ROUGE-1 → מילים
- ROUGE-2 → זוגות מילים
- ROUGE-L → רצפים

💠 (12 baseline (חשוב למבחן!)

Python

```
from nltk.tokenize import sent_tokenize

:(def three_sentence_summary(text

([return "\n".join(sent_tokenize(text))[:3
```

למה? 🧠

👉 benchmark פשוט להשוואה

💠 (13 מודל

Python

```
from transformers import AutoModelForSeq2SeqLM
```

```
model =  
(AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint
```

Data Collator (14 ♦

Python

```
from transformers import DataCollatorForSeq2Seq  
  
)data_collator = DataCollatorForSeq2Seq  
  
,tokenizer  
  
model=model  
  
(
```

מה הוא עושה? 🧠

- input-ל padding
- labels-ל padding
- labels padded עם 100-
- יוצר decoder_input_ids

קריטי 🔥

decoder_input_ids = labels shifted right

compute_metrics (15 )

Python

```
import numpy as np

from nltk.tokenize import sent_tokenize

:(def compute_metrics(eval_pred

    predictions, labels = eval_pred

)decoded_preds = tokenizer.batch_decode

    ,predictions

    skip_special_tokens=True

(

labels = np.where(labels != -100, labels,
                  (tokenizer.pad_token_id
```

```

)decoded_labels = tokenizer.batch_decode

,labels

skip_special_tokens=True

(

] = decoded_preds

(((n".join(sent_tokenize(pred.strip\

for pred in decoded_preds

[

] = decoded_labels

(((n".join(sent_tokenize(label.strip\

for label in decoded_labels

[

)result = rouge_score.compute

```



```

        ,predictions=decoded_preds

        ,references=decoded_labels

        use_stemmer=True

    (

    } = result

    key: value.mid.fmeasure * 100

    ()for key, value in result.items

    {

    {}

    {()return {k: round(v, 4) for k, v in result.items

```

Training (16 

Python

```

from transformers import Seq2SeqTrainingArguments

```

```
)args = Seq2SeqTrainingArguments  
  
    , "model"  
  
    , "evaluation_strategy="epoch  
  
    , learning_rate=5.6e-5  
  
    , per_device_train_batch_size=8  
  
    , per_device_eval_batch_size=8  
  
    , weight_decay=0.01  
  
    , num_train_epochs=8  
  
    , predict_with_generate=True  
  
    (
```

חשוב 🔥

Python

```
predict_with_generate=True
```

משתמש ב-generate 👉
כמו inference אמיתי 👉

Trainer ♦

Python

```
from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer

, model

, args

, ["train_dataset=tokenized_datasets["train

, ["eval_dataset=tokenized_datasets["validation

, data_collator=data_collator

, tokenizer=tokenizer

, compute_metrics=compute_metrics

(
```

אימון ♦

Python

```
(trainer.train
```

Python

```

from transformers import pipeline

summarizer = pipeline(
    "summarization",
    model="your-model"

)

"...summarizer("Long review text

```

מה קורה במודל? 🔥

בזמן training:

- משתמש ב-summary labels אמיתי)

בזמן inference:

- משתמש ב:

Python

generate()

• מייצר summary token-by-token

נושא	רעיון
task	sequence → sequence
input	טקסט ארוך
output	summary
labels	title
model	mT5
metric	ROUGE
baseline	3 משפטים
collator	padding + decoder
generate	inference

פרק 7.6

= Causal Language Model
מודל שלומד:

👉 לנחש את המילה הבאה לפי מה שכבר נכתב

דוגמה:

None

I love machine → learning

זה נקרא: 🧠

Auto-regressive model 👉

כי:

- כל token תלוי בקודמים



זה בדיוק כמו:

autocomplete 👉

כמו:

- ChatGPT
- Copilot
- השלמה אוטומטית בקוד

הבדל מ-MLM 🔥

הבדל מ-MLM 🔥

CLM	MLM
מנחש את הבא	מנחש באמצע
חד-כיווני	bidirectional

3) למה לא fine-tune? 💡

פה עושים:

TRAIN FROM SCRATCH 👉

מתי עושים את זה? 🧠

כש:

- יש הרבה דאטה
- הדאטה מאוד שונה

דוגמאות:

- קוד
- DNA
- מוזיקה

CodeParrot — דאטה (4 ♦

דאטה של קוד Python מ-GitHub

הבעיה 💡

הדאטה עצום (180GB)

פתרון 🔥

מסננים רק:

Python

```
["pandas", "sklearn", "matplotlib", "seaborn"]
```

(5 ♦ פונקציית סינון

Python

```
:(def any_keyword_in_string(string, keywords
```

```
:for keyword in keywords

:if keyword in string

    return True

    return False
```

הסבר 🧠

בודק אם קוד שייך ל-data science stack

♦ (6 יצירת dataset מסונן

Python

```
from collections import defaultdict

from datasets import Dataset

:(def filter_streaming_dataset(dataset, filters

    (filtered_dict = defaultdict(list

        :for sample in dataset
```



```

if any_keyword_in_string(sample["content"],
                           :(filters

:())for k, v in sample.items

(filtered_dict[k].append(v

(return Dataset.from_dict(filtered_dict

```

dataset טעינת (7 ♦

Python

```

from datasets import load_dataset

ds_train =
load_dataset("huggingface-course/codeparrot-ds-train",
              ("split="train

ds_valid =
load_dataset("huggingface-course/codeparrot-ds-valid",
              ("split="validation

```

8) בעיה מרכזית ♦

✗ קבצים ארוכים מאוד

פתרון 💡

Chunking 👉

Tokenization + Chunking (9 ⬢

Python

```
from transformers import AutoTokenizer

context_length = 128

tokenizer = AutoTokenizer.from_pretrained
("huggingface-course/code-search-net-tokenizer"

(

:(def tokenize(element

)outputs = tokenizer

,["element["content

, truncation=True

,max_length=context_length

,return_overflowing_tokens=True
```

```

        , return_length=True

    )

    [] = input_batch

for length, input_ids in zip(outputs["length"],
                               outputs["input_ids"]):

    if length == context_length:

        (input_batch.append(input_ids

{return {"input_ids": input_batch

```

הסבר עמוק 🧠

למה chunking?

כי:

- GPT מקבל אורך קבוע

למה זורקים קטעים קצרים?

כי:

- אין מספיק מידע
- לא רוצים padding

10 ♦ יצירת dataset סופי

Python

```
tokenized_datasets = raw_datasets.map  
  
    , tokenize  
  
    , batched=True  
  
    remove_columns=raw_datasets["train"].column_names  
  
    (
```

11 ♦ GPT2 — מודל

Python

```
from transformers import AutoConfig, GPT2LMHeadModel  
  
config = AutoConfig.from_pretrained  
  
    , "gpt2"  
  
    , (vocab_size=len(tokenizer  
  
    , n_ctx=context_length
```

```
,bos_token_id=tokenizer.bos_token_id

,eos_token_id=tokenizer.eos_token_id

(

(model = GPT2LMHeadModel(config
```

חשוב 🧠

from_pretrained לא משתמשים ב-👉
מתחילים מאפס 👉

Data Collator (12 ♦

Python

```
from transformers import DataCollatorForLanguageModeling

tokenizer.pad_token = tokenizer.eos_token

)data_collator = DataCollatorForLanguageModeling

,tokenizer
```

mlm=False

(

הסבר 🧠

mlm=False = causal LM 👉

חשוב 🔥

labels = input_ids shifted internally

Training (13 ♦

Python

```
from transformers import TrainingArguments, Trainer

args = TrainingArguments

, "output_dir="codeparrot-ds

, per_device_train_batch_size=32

, per_device_eval_batch_size=32

, gradient_accumulation_steps=8
```

```
        ,num_train_epochs=1

        ,learning_rate=5e-4

        ,weight_decay=0.1

        ,"lr_scheduler_type="cosine

        ,fp16=True

        ,push_to_hub=True

        (

        )trainer = Trainer

        ,model=model

        ,tokenizer=tokenizer

        ,args=args

        ,data_collator=data_collator

        ,["train_dataset=tokenized_datasets["train

        ,["eval_dataset=tokenized_datasets["valid

        (
```

14 (14) אימון

Python

```
()trainer.train
```

15 (15) שימוש — pipeline

Python

```
from transformers import pipeline

(pipe = pipeline("text-generation", model="your-model

(pipe("df = pd.DataFrame
```

מה קורה? 🧠

המודל משלים קוד 👉

16 (16) איך loss עובד

המודל לומד: 👉

None

$(\text{token}_t \rightarrow \text{token}_{t+1})$

חשוב 🔥

- $\text{label} = \text{token}$ הבא
- shift קורה בתוך המודל

Accelerate (17) ♦ (מתקדם)

נותן:

שליטה מלאה על training loop 👉

שלבים:

1. DataLoader
2. optimizer
3. scheduler
4. training loop
5. evaluation
6. save

custom loss (18) ♦ (מתקדם מאוד)

Python

```
def keytoken_weighted_loss(inputs, logits,  
                           :(keytoken_ids
```

```
    [:shift_labels = inputs[..., 1
```

```
    [:, shift_logits = logits[..., :-1
```

```
loss = CrossEntropyLoss()(shift_logits.view(-1,
                                              ((shift_logits.size(-1)
                                              ((shift_labels.view(-1
```

`return loss`

רעיון 🧠

👉 נותנים משקל גבוה למילים חשובות (כמו pandas)

👉 מה ההבדל בין MLM ל-CLM?

MLM מנבא מילים חסרות מתוך הקשר דו-כיווני, בעוד CLM מנבא את הטוקן הבא בצורה חד-כיוונית ולכן מתאים ליצירת טקסט.

נושא	רעיון
CLM	לנבא token הבא
GPT	autoregressive
input	קוד/טקסט
output	המשך
chunking	חובה
labels	shifted
collator	mlm=False
training	from scratch

סיכום על כל הקורס (שורה אחת לכל נושא) 🔥

נושא	רעיון
Token Classification	label לכל מילה
MLM	לנחש מילים חסרות
Translation	שפה → שפה
Summarization	טקסט → תקציר
CLM	לייצר טקסט

פרק 7.7

= Question Answering
נותנים:

- שאלה
- טקסט (context)

👉 והמודל צריך למצוא את התשובה בתוך הטקסט

דוגמה:

None

... Context: ... Mary appeared to Saint Bernadette

?Question: To whom did Mary appear

Answer: Saint Bernadette

◆

זה לא:

👉 לייצר תשובה

זה:

👉 למצוא span (קטע) בתוך הטקסט

🔥 זה נקרא:

Extractive QA

👉 התשובה תמיד נמצאת בתוך ה-context

◆ (3) איך המודל חושב?

במקום להגיד תשובה ישירות:

👉 הוא מנבא:

- start position

end position •

דוגמה 🧠

None

```
... tokens: [CLS] ... Saint Bernadette Soubirous
```

המודל מחזיר:

None

```
start = 10
```

```
end = 12
```

SQuAD — דאטה 4 ♦

Python

```
from datasets import load_dataset
```

```
("raw_datasets = load_dataset("squad
```

מבנה 🧠

Python

```
}
```

```
, "...": "context"
```

```

        ..., "question"

    } : "answers"

    ..., [...] : "text"

    [...] : "answer_start"

}

{

```

חשוב 🔥

- `answer_start` = מיקום בתווים
 - צריך להמיר ל-`token index`
-

tokenizer (5 ♦

Python

```

from transformers import AutoTokenizer

model_checkpoint = "bert-base-cased

tokenizer =
(AutoTokenizer.from_pretrained(model_checkpoint

```

6) איך נראה input?

None

[CLS] question [SEP] context [SEP]

7) בעיה: טקסט ארוך

פתרון: sliding window 💡

Python

```
inputs = tokenizer
    ,question
    ,context
    ,max_length=384
    ,"truncation="only_second
    ,stride=128
    ,return_overflowing_tokens=True
    (
```

הסבר 🧠

• חותך context לחלקים

- יש overlap ביניהם •

8) בעיה קשה: labels ♦

צריך לחשב: 🔥

None

(start_token, end_token)

9) offset mapping ♦

Python

```
inputs = tokenizer
    ,question
    ,context
return_offsets_mapping=True
(
```

מה זה? 🧠

ממפה:

None

(token → (start_char, end_char

10 ♦ חישוב start/end

Python

```
[start_char = answer["answer_start"][0]
([end_char = start_char + len(answer["text"][0]
```

ראו:

1. מוצאים איפה ה-context בטוקנים
2. בודקים אם התשובה נמצאת ב-chunk
3. אם לא $\rightarrow (0,0)$
4. אם כן $\rightarrow$ מוצאים token start/end

11 ♦ פונקציה מלאה (TRAIN)

Python

```
:(def preprocess_training_examples(examples
    questions = [q.strip() for q in
                  ["examples["question
    inputs = tokenizer
    , questions
    , ["examples["context
```

```

        ,max_length=384

        ,"truncation="only_second

        ,stride=128

    ,return_overflowing_tokens=True

    ,return_offsets_mapping=True

    ,"padding="max_length

    (

("offset_mapping = inputs.pop("offset_mapping

        sample_map =
("inputs.pop("overflow_to_sample_mapping

["answers = examples["answers

[] = start_positions

[] = end_positions

```

```

:(for i, offset in enumerate(offset_mapping

    [sample_idx = sample_map[i

    [answer = answers[sample_idx

[start_char = answer["answer_start"][0

([end_char = start_char + len(answer["text"][0

(sequence_ids = inputs.sequence_ids(i

    find context #

    idx = 0

:while sequence_ids[idx] != 1

    idx += 1

    context_start = idx

:while sequence_ids[idx] == 1

```

```

        idx += 1

        context_end = idx - 1

        answer not in chunk #

if offset[context_start][0] > start_char or
    :offset[context_end][1] < end_char

    (start_positions.append(0

    (end_positions.append(0

                                :else

                                start #

                                idx = context_start

:while offset[idx][0] <= start_char

                                idx += 1

    (start_positions.append(idx - 1

                                end #

```

```

        idx = context_end

        while offset[idx][1] >= end_char

            idx -= 1

        (end_positions.append(idx + 1

inputs["start_positions"] = start_positions

inputs["end_positions"] = end_positions

return inputs

```

validation preprocessing (12 ♦

Python

```

:(def preprocess_validation_examples(examples

    )inputs = tokenizer

    ,["examples["question

    ,["examples["context

    ,max_length=384

```

```

        , "truncation="only_second

        , stride=128

    , return_overflowing_tokens=True

    , return_offsets_mapping=True

    , "padding="max_length

    (

        sample_map =
        ("inputs.pop("overflow_to_sample_mapping

        [] = example_ids

        :(((["for i in range(len(inputs["input_ids

        [sample_idx = sample_map[i

        ([example_ids.append(examples["id"][sample_idx

```

```

(sequence_ids = inputs.sequence_ids(i
[offsets = inputs["offset_mapping"][i

] = [inputs["offset_mapping"][i

o if sequence_ids[k] == 1 else None

(for k, o in enumerate(offsets

[

inputs["example_id"] = example_ids

return inputs

```

מודל (13 ♦

Python

```

from transformers import AutoModelForQuestionAnswering

```

```
model =  
AutoModelForQuestionAnswering.from_pretrained(model_chec  
(kpoint
```

Training (14 ♦

Python

```
from transformers import TrainingArguments, Trainer  
  
args = TrainingArguments  
    , "bert-finetuned-squad"  
    , learning_rate=2e-5  
    , num_train_epochs=3  
    , weight_decay=0.01  
    , fp16=True  
  
(  
  
    )trainer = Trainer
```



```
        ,model=model
        ,args=args
        ,train_dataset=train_dataset
        ,eval_dataset=validation_dataset
        ,tokenizer=tokenizer
    )

    ()trainer.train
```

15 ♦ פלט המודל

המודל מחזיר:

Python

```
start_logits
end_logits
```

16 ♦ post-processing (קריטי!)

איך מוצאים תשובה?

1. לוקחים top start tokens
2. top end tokens
3. יוצרים combinations
4. מסננים:
 - לא בתוך context
 - $end < start$
 - ארוך מדי

קוד:

Python

```
[:start_indexes = np.argsort(start_logits)[-20]

[:end_indexes = np.argsort(end_logits)[-20]

    :for s in start_indexes

    :for e in end_indexes

        :if e < s

            continue

[[text = context[offsets[s][0]:offsets[e][1]
```

Python

```
import evaluate

metric = evaluate.load("squad"
```

מודדים: 

- Exact Match •
- F1 •

שימוש (18 ♦)

Python

```
from transformers import pipeline

qa = pipeline("question-answering", model="your-model")

qa(
    "?question="Where
    "context="The event was in Paris
    (
```

נושא	רעיון
QA	למצוא תשובה בתוך טקסט
input	question + context
output	start/end
sliding window	לטקסט ארוך
offset mapping	token → char
labels	start/end index
metric	EM + F1

משימה	סוג
Token Classification	token לכל label
MLM	ניחוש מילים
Translation	seq → seq
Summarization	seq → seq
CLM	generate
QA	span extraction

פרק 7.8

מעבר מ-NLP ל-LLMs 📌

בעבר:

עבדנו עם משימות ספציפיות (NER, QA וכו') 👉

היום:  יש LLMs (Large Language Models) שעושים הרבה דברים ביחד

מה זה בעצם LLM? 

מודל שפה גדול (כמו GPT, T5 וכו') שמאומן על הרבה דאטה ויכול:

- להבין טקסט
- לייצר טקסט
- לבצע משימות שונות

מה מיוחד ב-LLMs? 
1. עושים כמה משימות בלי fine-tuning

בעבר:

None

מודל לכל משימה

היום:

None

מודל אחד → הרבה משימות

2. מבינים הוראות (Instructions)

לדוגמה:

None

Translate this sentence to French

 המודל מבין מה רוצים ממנו

3. יודעים לייצר טקסט טוב

- תשובות
- סיכומים
- קוד
- סיפורים

4. יודעים "לחשוב" (reasoning)

באמצעות:
Chain-of-Thought 👉

כלומר:

None

Step 1 → Step 2 → Step 3

כל מה שלמדת קודם = הבסיס ל-LLMs 🧠	
נושא שלמדת	איך זה קשור ל-LLMs
Tokenization	איך המודל רואה טקסט
Transformer	הארכיטקטורה
Fine-tuning	התאמה לדאטה
Metrics	איך מודדים ביצועים
Data collators	איך מכינים דאטה

כלומר: 👉
LLMs = הרחבה של כל מה שכבר למדת

סוגי מודלים (קריטי!)
1. Encoder (כמו BERT)

טוב ל:

- classification
- (QA (extractive

2. Decoder (כמו GPT)

טוב ל:

- generation
- chat
- code

3. Encoder-Decoder (כמו T5)

טוב ל:

- translation
- summarization

👉 איזה מודל מתאים למה?

Pretraining vs Fine-tuning ♦
Pretraining

- לומד שפה כללית
- הרבה דאטה

Fine-tuning

- מתאימים למשימה ספציפית

♦ איך מאמנים מודלים

יש 2 דרכים:

Trainer API ✓

קל לשימוש
מעט קוד

Accelerate ✓

שליטה מלאה
יותר מורכב

♦ Metrics — חשוב להבין מגבלות

בעיה	Metric
לא מבין משמעות	BLEU
רק overlap	ROUGE
לא מתאים ל-generation	Accuracy

👉 כלומר:
metrics לא מושלמים!

♦ שימוש במודלים

אפשר:

Hub דרך ✓

pipeline דרך ✓

Python

```
("pipeline("task
```

מה באמת למדת מכל הקורס ✦

אֵיךְ לקחת דאטה → להפוך ל-tokens → לאמן מודל → להעריך → להשתמש 👉

זה ה-flow הכי חשוב: 🔥

None

Data → Tokenization → Model → Training → Evaluation → Inference

התמונה הגדולה ✦

בעבר:

פתרנו משימות NLP אחת אחת 👉

היום:

משתמשים ב-LLMs שעושים הכל 👉

אבל:

הם עדיין בנויים על אותם עקרונות! 👉

רעיון	משמעות
LLM	מודל כללי להרבה משימות
Instruction	המודל מבין מה לבקש
Reasoning	chain-of-thought
Architectures	encoder / decoder / seq2seq
בסיס	כל מה שלמדת קודם

👉 איך LLMs קשורים ל-NLP?

LLMs הם הרחבה של מודלי NLP מסורתיים, המבוססים על אותם עקרונות כמו tokenization ו-Transformers, אך עם יכולת לבצע מגוון משימות באמצעות מודל אחד במקום מודלים נפרדים לכל משימה.



כל מה שלמדת (Token, MLM, Translation, QA וכו') הם פשוט שימושים שונים של אותו מודל Transformer — ו-LLMs מאחדים את כולם למודל אחד חזק.

פרק 7.9

שאלה 1

EN: Which of the following tasks can be framed as a token classification problem?
HE: אילו מהמשימות הבאות ניתן להגדיר כ-Token Classification?

A.

Find the grammatical components in a sentence
למצוא את הרכיבים הדקדוקיים במשפט

B.
Find whether a sentence is grammatically correct or not
לקבוע האם משפט תקין דקדוקית

C.
Find the persons mentioned in a sentence
למצוא אנשים שמוזכרים במשפט

D.
Find the chunk of words in a sentence that answers a question
למצוא את קטע המילים שעונה על שאלה

שאלה 2

EN: What part of the preprocessing for token classification differs from the other preprocessing pipelines?
HE: מה שונה ב-preprocessing של Token Classification לעומת משימות אחרות?

A.
There is no need to do anything; the texts are already tokenized
לא צריך לעשות כלום

B.
The texts are given as words, so we only need to apply subword tokenization
הטקסט כבר מחולק למילים

C.
We use -100 to label the special tokens
משתמשים ב--100 לטוקנים מיוחדים

D.
We need to make sure to truncate or pad the labels to the same size as the inputs
צריך להתאים labels לאורך של ה-input

שאלה 3

EN: What problem arises when we tokenize words in token classification?
HE: מה הבעיה כשעושים tokenization?

A. tokenizer מוסיף special tokens ואין להם labels
B. כל מילה יכולה להפוך לכמה tokens → יש יותר tokens מ-labels
C. אין בעיה

שאלה 4

EN: What does "domain adaptation" mean?
HE: מה זה domain adaptation?

- A. להריץ מודל
- B. לאמן מודל
- C. לבצע fine-tuning לדאטה חדש
- D. להוסיף טעויות לדאטה

שאלה 5

EN: What are the labels in MLM?
HE: מה הם ה-labels ב-MLM?

- A. tokens מקוריים אחרי masking
- B. shifted tokens
- C. sentiment
- D. similarity

שאלה 6

EN: Which tasks are sequence-to-sequence?
HE: אילו משימות הן seq2seq?

- A. כתיבת סיכומים
- B. QA
- C. תרגום
- D. תיקון טקסט

שאלה 7

EN: How to preprocess seq2seq?
HE: איך עושים preprocessing?

- A. inputs+targets יחד
- B. בנפרד
- C. רק inputs
- D. context manager

שאלה 8

EN: Why special Trainer for seq2seq?
HE: למה יש Trainer מיוחד?

- A. loss
- B. evaluation
- C. targets טקסט
- D. 2 models

שאלה 10

EN: When should you pretrain a new model?
HE: מתי צריך pretraining?

- A. אין מודל לשפה
- B. יש הרבה דאטה
- C. bias
- D. מודלים לא טובים

שאלה 11

EN: Why is pretraining easy?
HE: למה קל pretraining?

- A. יש הרבה דאטה
- B. לא צריך labels
- C. קל בקוד

שאלה 12

EN: QA preprocessing challenges?
HE: מה האתגרים?

- A. tokenize
- B. context ארוך
- C. tokenize תשובה
- D. למצוא start/end

שאלה 13

EN: QA post-processing?
HE: איך עושים post-processing?

- A. decode span
- B. לבחור feature הכי טוב

C. למפות ל-context
D. generate תשובה

שאלה	תשובות נכונות
1	A, C
2	C, D
3	A, B
4	C
5	A
6	A, C, D
7	A
8	B
10	A, B, C, D
11	A, B
12	B, D
13	B, C

שאלה 1

Token classification = label לכל מילה

✓ דקדוק / ישויות → כן

✗ משפט תקין → classification רגיל

שאלה 2

הייחוד:

- 100 - ל-special tokens
 - labels צריכים להיות באותו אורך
-

שאלה 3

בעיה מרכזית:

tokenizer מפצל מילים → mismatch 🙅

שאלה 4

Domain adaptation:

fine-tuning 🙅 על דאטה חדש

שאלה 5

:MLM

מנבאים את הטוקן המקורי 🙅

שאלה 6

Seq2Seq:

input → output 🙅 טקסט

summarization ✓

translation ✓

text correction ✓

QA = לא תמיד (span) ✗

שאלה 7

הנכון:

(tokenizer(inputs, text_target=targets) 🙅

שאלה 8

Trainer מיוחד:

evaluation שונה (generate) 🙅

שאלה 10

:Pretraining

👉 כשאין מודל / הרבה דאטה / בעיות במודל

שאלה 11

קל כי:

👉 לא צריך labeling

שאלה 12

QA קשה כי:

- context ארוך
- start/end צריך

שאלה 13

:Post-processing

👉 בוחרים span הכי טוב מתוך כמה chunks

פרק 8

פרק 8.1

עכשיו כשאת יודעת לעבוד עם כל המשימות ב-Transformers, את יכולה להתחיל פרויקטים אמיתיים.

אבל—

👉 תמיד תיתקלי בבעיות !

ולכן הפרק הזה מלמד:

- מה לעשות כשיש שגיאה
- איך לעשות debugging
- איך לבקש עזרה נכון

• איך לדווח על באגים

2 ♦ הסבר עמוק + אינטואיציה
למה הפרק הזה קיים בכלל? 🧠

כי בעולם האמיתי:

80% מהעבודה זה לא מודלים 👉
זה לפתור בעיות 😊👉

אינטואיציה 💡

תחשבי על זה ככה:

יש לך pipeline:

None

Data → Tokenizer → Model → Training → Evaluation

אם משהו נשבר ❌

את חייבת לדעת איפה ולמה 🙌

3 ♦ מה לומדים בפרק (הכי חשוב)

1. ✓ מה עושים כשיש שגיאה

לא לרוץ לקוד 🙌

לא לנחש 🙌

קודם: 🙌

להבין את השגיאה

2. ✓ איך לבקש עזרה

לא:

None

"לא עובד לי"

כן:

None

קוד + שגיאה + מה ניסיתי

✓ 3. איך עושים debugging

👉 לפרק את הבעיה לשלבים

במקום:

None

הכל לא עובד

לעשות:

None

tokenizer עובד?

model עובד?

inputs תקינים?

✓ 4. איך כותבים issue טוב

👉 כדי שמישהו יוכל לעזור לך מהר

4 🔥 הרעיון הכי חשוב בפרק

Debugging = הבנה של pipeline 🙌

למה? 🧠

כי כל הבעיות מגיעות מאחד מאלה:	
שלב	בעיה טיפוסית
Data	פורמט לא נכון
Tokenizer	mismatch
Model	shape error
Training	loss לא יורד

5 💠 החיבור למה שלמדת קודם

כל הנושאים הקודמים = מקור לבאגים	
נושא	בעיה קלאסית
Token Classification	label mismatch
MLM	masking לא נכון
Translation	targets לא נכון
QA	start/end לא נכון
CLM	shift לא נכון

כלומר: 🙌

הפרק הזה מסביר איך לפתור את כל הבעיות האלה

6) זה לא רק Hugging Face ♦

חשוב: 📌

זה לא רק Transformers

👉 זה נכון לכל פרויקט קוד

7) הדרך הנכונה לחשוב (סופר חשוב למבחן) ♦

כשיש בעיה:

❌ לא נכון:

- לנסות דברים אקראיים
- להעתיק קוד

✅ נכון:

1. להבין מה קרה
2. לקרוא error
3. לצמצם את הבעיה
4. לבדוק שלבים

8) הסיכום הכי חשוב 🔥

👉 להיות טוב ב-ML = להיות טוב ב-debugging

פרק 8.2

בקטע הזה מראים:

👉 אילו שגיאות נפוצות קורות כשמתמשים במודל
👉 ואיך לפתור אותן

דרך דוגמה אמיתית של QA model

♦ (2) האינטואיציה הכי חשובה
🧠 מה בעצם קורה כאן?

יש לך מודל
את מנסה להריץ pipeline:

Python

```
from transformers import pipeline

reader = pipeline("question-answering",
                  (model=model_checkpoint
```

💥 בום — שגיאה

👉 וזה מצב אמיתי לגמרי בעולם ML

🔥 הרעיון המרכזי

👉 Debugging = לקרוא נכון את השגיאה

3) הבעיה הראשונה — model לא נטען
השגיאה: ❌

None

`OSError: Can't load config`

איך חושבים נכון? 🧠

שלב 1:

👉 לא להיבהל

שלב 2:

👉 לקרוא את השגיאה

🔥 כלל סופר חשוב

👉 traceback קוראים:

מלמטה למעלה

למה?

כי:

- למטה = השגיאה האמיתית
- למעלה = קריאות פונקציות

4) מה למדנו מהשגיאה? ♦

השגיאה אומרת:

👉 בעיה ב־`config.json`
👉 או ID model לא נכון

💡 שלב חשיבה

👉 לבדוק model ID

✖ טעות:

None

distillbert

✓ נכון:

None

distilbert

🔥 מסקנה

👉 שגיאות קטנות (typo) = בעיות גדולות

🔹 (5) הבעיה השנייה — חסר config.json

גם אחרי תיקון השם:

💥 עדיין שגיאה

🧠 מה עושים?

בודקים repo:

Python

```
from huggingface_hub import list_repo_files
```

```
(list_repo_files(repo_id=model_checkpoint
```

מה גלינו? 💡

אין config.json 🙅

למה זה קריטי? 🔥

כי: 🙅
המודל לא יכול להיטען בלי config

6 פתרון 💎

לוקחים config מהמודל המקורי:

Python

```
from transformers import AutoConfig

config =
("AutoConfig.from_pretrained("distilbert-base-uncased

(config.push_to_hub(model_checkpoint
```

אינטואיציה 🧠

המודל צריך: 🙅

- weights •
- tokenizer •

• config

🔥 ואז זה עובד:

Python

```
reader = pipeline("question-answering",  
                  (model=model_checkpoint
```

🔹 (7 חלק שני — Debugging ב-forward pass

❌ שגיאה:

None

```
'AttributeError: 'list' object has no attribute 'size
```

🧠 מה זה אומר?

👉 המודל מצפה ל-tensor
👉 אבל קיבל list

💡 למה?

כי עשינו:

Python

```
(inputs = tokenizer(question, context
```

❌ זה מחזיר:

Python

`list`

אבל המודל צריך: 🔥

Python

`tensor`

8 הפתרון ♦

Python

```
inputs = tokenizer
,question
,context

"return_tensors="pt

(
```

אינטואיציה 🧠

מודלים עובדים על tensors
לא על Python lists

9 הקוד המלא (עובד) ♦

Python

```
import torch

inputs = tokenizer

,question

,context

,add_special_tokens=True

"return_tensors="pt

(

[input_ids = inputs["input_ids"]][0

(outputs = model(**inputs

answer_start_scores = outputs.start_logits

answer_end_scores = outputs.end_logits
```

```

(answer_start = torch.argmax(answer_start_scores

answer_end = torch.argmax(answer_end_scores) + 1

)answer = tokenizer.convert_tokens_to_string

tokenizer.convert_ids_to_tokens(input_ids[answer_start:a
([answer_end

(

(print(answer

```

10) מה למדנו מכל הסיפור 

1. לקרוא traceback נכון 

- מלמטה למעלה
- להבין error type

2. לבדוק דברים בסיסיים 

- model name
- files
- config

3. להבין types 🔥

• list ≠ tensor

4. להשתמש בגוגל / StackOverflow 🔥

👉 זה חלק מהעבודה!

♦ (11) תהליך Debug נכון (הכי חשוב למבחן)

תמיד לעבוד ככה:

1. לקרוא error
2. להבין מה מצופה
3. לבדוק input
4. לבדוק types
5. לבדוק repo

👉 איך ניגשים לשגיאה?

קוראים את ה-traceback מהסוף להתחלה, מזהים את סוג השגיאה, ואז בודקים את הקלטים והסוגים (types) כדי להבין איפה המודל קיבל ערך לא נכון.

פרק 8.3

הקטע מסביר:

👉 איך להשתמש בפורומים של Hugging Face
👉 ואיך לכתוב שאלה (topic) בצורה שתענה מהר

♦ (2) אינטואיציה עמוקה

🧠 מה הבעיה?

הרבה אנשים שואלים ככה:

None

"It doesn't work"

👉 ואף אחד לא עונה 😊

למה? 💡

כי: 👉
אי אפשר להבין מה הבעיה

הרעיון המרכזי 🔥

👉 כדי לקבל עזרה — צריך לאפשר לאחרים להבין את הבעיה שלך

♦ (3 איך הפורום עובד

מבנה 🧠

- (categories (Beginner / Intermediate / Research
- topics (שאלות)

בחירת קטגוריה💡	
מצב	קטגוריה
מתחילה	Beginners
ביניים	Intermediate
מחקר	Research
קורס	Course

💎 (4) איך כותבים שאלה טובה

🔥 1. כותרת טובה (קריטי!)

✖ גרוע:

None

!!!Help

✔ טוב:

None

IndexError in AutoModel forward pass

🧠 למה זה חשוב?

כ"י:

אנשים מחפשים לפי כותרת 👉

2. פורמט קוד נכון 🔥

גרוע: ❌

קוד רגיל

טוב: ✅

Python

```
(...)model = AutoModel.from_pretrained
```

איך עושים? 🧠

None

```
```code```
```

למה? 💡

כדי שיהיה קריא 👉

3. להביא את כל ה-`traceback` 🔥

טעות: ❌

רק השורה האחרונה



---

נכון: ✓

כל השגיאה 🙌

---

למה? 🧠

כי: 🙌  
לפעמים המידע למעלה חשוב

---

4. דוגמה שניתן להריץ (הכי חשוב!) 🔥

---

גרוע: ✖

None

יש לי בעיה

---

טוב: ✓

Python

```
(inputs = tokenizer(text

(outputs = model(**inputs
```

---

אינטואיציה 🧠

מי שעוזר לך צריך לשחזר את הבעיה 🙌

---

5) הדוגמה מהקטע ♦

---

שגיאה: ❌

None

IndexError: index out of range

---

מה קרה? 🧠

הטקסט ארוך מדי:

None

512 < 583

---

הבעיה 🔥

עברנו את max\_length של המודל 🙌

---

פתרון (אינטואיטיבי) 💡

- לחתוך טקסט
- או להשתמש ב-truncation

---

6) הטעות הכי נפוצה 💎

לא נותנים מספיק מידע 🙌

---

למה זה קורה? 🧠

כי חושבים:

"זה ברור" 🙌

אבל זה לא 😊

---

## 7) התהליך הנכון לבקשת עזרה

---

תמיד:

1. לכתוב כותרת ברורה
2. להוסיף קוד
3. להוסיף error
4. להוסיף traceback
5. לתת דוגמה מינימלית

---

זה נקרא: 🔥

Reproducible Example 🙌

---

## 8) החיבור למה שלמדת

---

כל הבאגים שלך = קשורים לזה

נושא	בעיה
QA	context ארוך
MLM	masking
Token	labels
CLM	tensors

## פרק 8.4

יש לך קוד יפה לאימון מודל  
ואת מריצה:

Python

```
trainer.train()
```

יש שגיאה 🌟  
או יותר גרוע:  
אין שגיאה אבל המודל גרוע 🤖

---

המטרה של הקטע: 🙌  
להראות איך למצוא את הבעיה

2) האינטואיציה הכי חשובה 🔥  
מה הבעיה הגדולה? 🧠

`trainer.train()` עושה מלא דברים:

None

Dataset → Dataloader → Model → Loss → Optimizer →  
Evaluation

---

ולכן: 🙌  
הבעיה יכולה להיות בכל שלב

---

כלל זהב 🔥



Debugging = לפרק את ה-pipeline לשלבים

◆ (3) שלב 1 — לבדוק את הדאטה

❌ שגיאה:

None

```
ValueError: specify input_ids
```

🧠 מה קרה?

הדאטה היה:

Python

```
{'...': 'premise': '...', 'hypothesis'}
```

👉 טקסטים  
👉 לא tokenized

❌ טעות

השתמשנו ב:

Python

```
train_dataset = raw_datasets
```

✅ נכון

Python

```
train_dataset = tokenized_datasets
```

---

מסקנה 🔥

תמיד לבדוק: 🙌

Python

```
[trainer.train_dataset[0]
```

---

tokenize שלב 2 — לבדוק 🔹

---

איך בודקים? 🧠

Python

```
("tokenizer.decode(trainer.train_dataset[0]["input_ids
```

---

זה נותן לך טקסט אמיתי 🙌

---

למה זה חשוב? 💡

כי: numbers לא אומרים כלום 🙌

---

5) שלב 3 — לבדוק מבנה הנתונים 🔹

---

לבדוק keys:

Python

```
()trainer.train_dataset[0].keys
```

---

צריך לראות: 🙌

- input\_ids
- attention\_mask
- labels

---

חשוב 🔥

דברים שלא תואמים למודל → נחקים אוטומטית

---

DataLoader — 4 שלב 6 ♦

---

איך בודקים? 🧠

Python

```
:()for batch in trainer.get_train_dataloader
```

```
break
```

---

אם יש שגיאה → הבעיה פה 🙌

---

שגיאה: ❌

None

expected sequence length mismatch

למה? 🧠

כי:  
padding 🙌

collator לא נכון: ❌

Python

default\_data\_collator

פתרון: ✅

Python

```
from transformers import DataCollatorWithPadding
```

```
data_collator =
(DataCollatorWithPadding(tokenizer=tokenizer
```

מסקנה 🔥

batching = מקור להרבה באגים 🙌



7 שלב 5 — לעבור דרך המודל

לוקחים batch: 🧠

Python

```
(outputs = model(**batch
```

שגיאה: ❌

None

CUDA error

כלל חשוב 🔥

👉  
CPU → CUDA errors לעבור ל-CPU

פתרון: ✅

Python

```
(outputs = model.cpu(**batch
```

למה? 🧠?

:GPU errors

- לא מדויקים
- קשים להבנה

8 בעיה אמיתית שהתגלתה

---

שגיאה: ❌

None

IndexError: target 2 out of bounds

---

למה? 🧠

:dataset

None

labels = 0,1,2

אבל המודל:

Python

num\_labels = 2

---

פתרון: ✅

Python

```
model =
)AutoModelForSequenceClassification.from_pretrained

,model_checkpoint

num_labels=3

(
```

מסקנה 🔥

labels mismatch = באג קלאסי 🙌

optimizer — 6 שלב 9 ♦

עושים: 🧠

Python

```
()loss.backward
```

```
()trainer.optimizer.step
```

לרוב אין בעיות פה 🙌  
אבל אם כן → שוב CPU

CUDA Out Of Memory 10 ♦

שגיאה: ❌

None

CUDA out of memory

זה אומר: 🧠

אין מקום ב-GPU 🙌

פתרונות: ✓

- להקטין batch size
- להשתמש במודל קטן יותר
- לוודא שאין 2 מודלים בזיכרון

evaluation — 7 שלב 11 ♦

שגיאה: ✗

None

`TypeError: only size-1 arrays`

למה? 🧠

predictions = logits  
אבל metric רוצה labels

פתרון: ✓

Python

```
(predictions = np.argmax(predictions, axis=1
```

מסקנה 🔥

logits ≠ predictions 🙅

debugging בלי שגיאה (הכי קשה) 12 ♦

---

מצב: 🤖

אין error  
אבל model גרוע

---

מה עושים? 🔥

---

✓ לבדוק דאטה

- האם הגיוני?
- labels נכונים?
- balanced?

---

✓ לבדוק predictions

👉 האם המודל תמיד מנבא אותו דבר?

---

✓ לבדוק loss

👉 האם הגיוני?

---

♦ (13 בדיקה סופר חשובה

---

OVERFIT על batch אחד 🔥

---

קוד:

Python

```
:()for batch in trainer.get_train_data_loader
```

```
break
```

```
:(for _ in range(20
```

```
(outputs = model(**batch
```

```
loss = outputs.loss
```

```
()loss.backward
```

```
()optimizer.step
```

---

מה מצפים? 🧠

accuracy 100% 👍

---

אם לא: ❌

יש בעיה בדאטה / מודל 👍

---

hyperparameters (14 💎

---

טעות: ❌

לשנות מיליון פרמטרים

---

נכון: ✅

👉 קודם baseline

---

♦ 15) תהליך Debug נכון (זהב למבחן)

---

תמיד:

1. לבדוק data
2. לבדוק tokenize
3. לבדוק batch
4. לבדוק model
5. לבדוק loss
6. לבדוק evaluation

## פרק 8.5

אם מצאת באג בספרייה (כמו Transformers):

👉 צריך לדווח עליו

— אבל

👉 כדי שיעזרו לך מהר  
צריך לכתוב Issue טוב

---

🔥 (2) האינטואיציה הכי חשובה  
🧠 מה הבעיה?

הרבה אנשים כותבים:

None

It doesn't work

👉 ואף אחד לא יכול לעזור

למה? 💡

כי:

👉 אי אפשר להבין מה הבעיה

רעיון מרכזי 🔥

👉

Issue טוב = מאפשר לאחרים לשחזר את הבעיה

♦ (3) שלב 1 — לבדוק אם זה באמת באג

לפני שכותבים issue: 🧠

👉 לבדוק בפורום

למה? 💡

כי:

- אולי זו טעות שלך
- אולי כבר פתרו

♦ (4) שלב 2 — Minimal Reproducible Example



הכי חשוב בפרק הזה 🔥

מה זה? 🧠

קוד קטן שמייצר את הבאג 🙌

גרוע: ❌

None

יש לי בעיה בפרויקט גדול

טוב: ✅

Python

```
(...)model = AutoModel

(outputs = model(inputs
```

# פה השגיאה

תנאים: 🔥

- self-contained
- בלי קבצים חיצוניים
- עובד לבד

טריק 💡

להחליף דאטה אמיתי ב-dummy 🙌

5 ♦ למה זה קריטי?

כ״: 🙌

אי אפשר לתקן מה שלא רואים

template — שלב 3 — למלא template

למה? 🧠

template בותן GitHub

🙌 לא להשאיר ריק!

מה למלא: 🔥

- מה קרה
- מה ציפית
- קוד
- שגיאה

environment (7 — חשוב מאוד)

קוד:

Shell

```
transformers-cli env
```

למה? 🧠

כ״: 🙌

באגים תלויים בגרסה

דוגמה:

- Python version
- PyTorch
- GPU

8 tagging אנשים ♦

מה זה? 🧠

None

username@

מתי להשתמש? 💡

- אם יודעים מי קשור

! כלל חשוב

👉 לא לתייג יותר מדי אנשים

9 לצרף קוד + שגיאה ♦

נכון: ✅

Python

# קוד

None

error #

למה? 🧠

readability 👉

expected behavior (10 💎

מה לכתוב? 🧠

מה רצית שיקרה 👉

דוגמה:

None

Expected: model runs without error

Got: ValueError

11) אחרי שפרסמת 💎

מה עושים? 🧠

- לבדוק שלא טעית
- לעדכן אם צריך

! לא לעשות:

👉 להציק לאנשים

💡 אם אין תשובה

👉 לשפר את הדוגמה

♦ (12 החיבור למה שלמדת

## כל הפרק מתחבר:

שלב	למה חשוב
Debugging	להבין בעיה
Forums	לשאול
Issue	לדווח

👉 זה flow:

None

Error → Debug → Ask → Issue

🔥 (13 הטעות הכי נפוצה

👉 לא עושים minimal example

למה זה קורה? 🧠

כי:

👉 "זה ברור לי"

אבל לאחרים זה לא

---

♦ (14) מה באמת חשוב לזכור

---

🧠 חוק זהב

👉

אם מישהו לא יכול להריץ את הקוד שלך — הוא לא יכול לעזור לך

---

## פרק 8.6

סיימת את חלק 2 של הקורס 🎉

👉 עכשיו את יודעת:

- לעבוד עם משימות NLP
- לעשות fine-tuning
- ואפילו pretraining

---

בנוסף:

👉 ממליצים:

- לשתף מודלים ב-Model Hub
- להמשיך לפרויקטים אמיתיים

---

מה המשמעות האמיתית של הסיום הזה? 🧠 ♦

זה לא סתם "סיימת קורס"

זה אומר: 👉

יש לך את כל הכלים לבנות מערכות NLP אמיתיות

מה זה כולל בפועל? 💡

את כבר יודעת:

None

Data → Tokenizer → Model → Training → Evaluation → Inference

זה הדבר הכי חשוב מכל הקורס 🔥

זה ה-pipeline השלם 👉

3 ♦ מה באמת למדת (עמוק)

שליטה במשימות NLP ✓

משימה	מה למדת
Token Classification	token לכל labels
MLM	לנחש מילים
Translation	שפה → שפה
Summarization	טקסט → תקציר
CLM	יצירת טקסט
QA	למצוא תשובה

שליטה בטכניקות ✓

- Tokenization
- Data Collators
- Metrics
- Trainer / Accelerate

---

שליטה ב-debugging ✓

(מהפרק האחרון!)

👉 שזה הכי חשוב בעולם האמיתי

---

4 ♦ המעבר האמיתי

---

לפני הקורס: 🧠

👉 "איך משתמשים במודל?"

---

אחרי הקורס:

👉 "איך בונים מערכת שלמה"

---

שינוי חשיבה 🔥



לפני	אחרי
להריץ קוד	להבין pipeline
מודל אחד	משימות שונות
copy-paste	debugging

5) למה חשוב לשתף מודלים

Model Hub 🧠

זה GitHub של מודלים

למה להשתמש? 💡

- שיתוף
- שיפור
- קהילה

זה חלק מהעולם האמיתי של ML 🙌

6) מה השלב הבא

הקורס אומר:

Part 3 בדרך 🙌

אבל בפועל:

👉 את כבר יכולה:

- לבנות פרויקטים
- להתכונן לראיונות
- לעבוד עם LLMs

---

♦ (7 התמונה הגדולה

---

🧠 כל הקורס מתכנס לזה:

👉 Transformers = כלי אחד שמבצע הרבה משימות

---

🔥 והשלב הבא:

👉 LLMs  
(שמאחדים הכל)

## פרק 8.7

שאלה 1

EN: In which order should you read a Python traceback?  
HE: באיזה סדר קוראים traceback בפיתון?

A.  
From top to bottom  
מלמעלה למטה

B.  
From bottom to top  
מלמטה למעלה

---

שאלה 2

EN: What is a minimal reproducible example?  
HE: מה זה minimal reproducible example?

A.  
A simple implementation of a Transformer architecture  
מימוש פשוט של Transformer

B.  
A compact and self-contained block of code that can run independently  
קוד קצר ועצמאי שניתן להרצה

C.  
A screenshot of the traceback  
צילום מסך

D.  
A full notebook  
מחברת מלאה

---

שאלה 3

EN: Which is a good forum topic title?  
HE: מה כותרת טובה לפורום?

A.  
Error message full copy  
העתקת השגיאה המלאה

B.  
Problem with import  
בעיה עם import

C.  
?Why can't I import GPT3ForSequenceClassification  
למה אני לא מצליח לייבא

D.  
?Is GPT-3 supported in 🤗 Transformers  
האם GPT-3 נתמך

---

שאלה 4

EN: First place to look for errors in training pipeline?  
HE: איפה מתחילים לחפש שגיאות?

A. optimization  
B. evaluation  
C. datasets  
D. dataloaders

---

שאלה 5

EN: Best way to debug CUDA error?  
HE: איך הכי טוב לדבג CUDA error?

- A. post online
- B. run on CPU
- C. read traceback
- D. reduce batch
- E. restart kernel

---

שאלה 6

EN: Best way to get GitHub issue fixed?  
HE: איך הכי טוב שיפתרו issue?

- A. full reproducible example
- B. ask every day
- C. inspect code

---

שאלה 7

EN: Why overfit one batch?  
HE: למה overfit על batch אחד?

- A. זה רע
- B. לבדוק שה-loss יורד ל-0
- C. לבדוק shapes

---

שאלה 8

EN: Why include environment info?  
HE: למה לצרף environment?

- A. גרסת ספרייה
- B. מערכת הפעלה
- C. GPU/CPU

## תשובות ✓

שאלה	תשובות נכונות
1	B
2	B
3	C, D
4	C
5	B
6	A, C
7	B, C
8	A, B, C

### שאלה 1

התשובה: מלמטה למעלה

כי: 👉

- השורה האחרונה = השגיאה האמיתית

### שאלה 2

התשובה: B

👉 קוד קצר שאפשר להריץ לבד

👉 בלי קבצים חיצוניים

### שאלה 3

תשובות: C, D

👉 כותרת טובה = מתארת את הבעיה

❌ לא לשים רק error  
❌ לא כללי מדי

---

שאלה 4

התשובה: datasets

👉 כי:

רוב הבאגים מתחילים מהדאטה

---

שאלה 5

התשובה: run on CPU

👉 :CUDA errors

- לא ברורים
- CPU נותן שגיאה מדויקת

---

שאלה 6

תשובות: A, C

👉 הכי חשוב:

- reproducible example
- להבין קוד

❌ לא להציק

---

שאלה 7

תשובות: B, C

👉 למה זה טוב?

- לבדוק שהמודל מסוגל ללמוד
- לבדוק שאין בעיות shapes

---

שאלה 8

תשובות: A, B, C

environment חשוב כי: 🙌

- גרסאות שונות → באגים שונים
  - GPU vs CPU
  - מערכת הפעלה
-